# ChatGPT

> compiled code like C++ don't run in all OS

Correct. A compiled C++ program does **not** automatically run on every operating system.

Here's why:

- **Source code (`.cpp`)** is portable. You can usually compile the same source code on Windows, Linux, or macOS with only minor changes.
- **Compiled executable** is **OS-specific** because it contains machine code and uses the operating system's binary format and system libraries.

For example:

| Compiled on | Output | Runs on |
|---|---|---|
| Windows | **`.exe`** | Windows only |
| Linux | ELF executable | Linux only |
| macOS | Mach-O executable | macOS only |

### Why?

Each operating system has:

- A different executable file format.
- Different system libraries and APIs.
- Different application binary interfaces (ABIs).

### How to make a C++ program run on multiple OSes?

You have a few options:

1. **Compile separately for each OS** (the most common approach).
2. **Cross-compile** (compile on one OS to produce an executable for another OS).
3. Write **portable C++ code** using the standard library and avoid OS-specific APIs when possible.

### Comparison with Java and Python

- **C++:** Compile for each target OS.
- **Java:** Compile once to bytecode; runs anywhere with a compatible JVM.
- **Python:** The same `.py` file runs on any OS with a compatible Python interpreter.

So, the **C++ source code can be portable, but the compiled executable is generally not.**

what is generic programming in C++ in understand and simple way

**Generic programming** means **writing code once so it can work with many different data types**.

In C++, this is mainly done using **templates**.

## Without generic programming

Suppose you want a function to find the larger of two numbers.

For integers:

```C++
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

For doubles, you would have to write another function:

```C++
double max(double a, double b) {
    return (a > b) ? a : b;
}
```

For characters, another one:

```C++
char max(char a, char b) {
    return (a > b) ? a : b;
}
```

This repeats the same logic.

---

## With generic programming

Using a template:

```C++
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}
```

Now the same function works for different types:

```C++
cout << max(10, 20);        // int
cout << max(5.5, 2.3);      // double
cout << max('A', 'Z');      // char
```

The compiler automatically creates the correct version for each type.

---

## Real-life example

Imagine a **water bottle mold** in a factory.

- One mold can make **small**, **medium**, or **large** bottles.
- The **shape** is the same; only the **size** changes.

Similarly, a C++ template is like a **blueprint**:

- The logic stays the same.
- The data type changes (`int`, `double`, `char`, etc.).

---

## Why use generic programming?

- ✅ Write code once.
- ✅ Reuse it for many data types.
- ✅ Reduce duplicate code.
- ✅ Easier to maintain.

---

## Simple definition for exams

> **Generic programming** is a programming technique in C++ that allows you to write a single function or class that works with different data types using **templates**, reducing code duplication and improving reusability.

**In one sentence:**
**Generic programming = "Write once, use with many data types."**

> what is high level abstractions

**High-level abstraction** means **hiding the complex implementation details and showing only the important features that a programmer needs to use**.

## Simple example

Think of a **TV remote**.

- You press the **Power** button to turn on the TV.
- You don't need to know how the electronic circuits inside the remote work.

The remote provides a **high-level abstraction**: a simple interface while hiding the complexity.

---

## In C++

Consider the `std::vector` container.

```
C++
```

```
vector<int> numbers;

numbers.push_back(10);
numbers.push_back(20);
```

You simply call `push_back()`.

You **don't need to know**:

- How memory is allocated.
- How the array grows when it becomes full.
- How elements are copied.

`vector` hides all those implementation details and provides an easy-to-use interface. That's a **high-level abstraction**.

---

## Another example

When you write:

```
C++
cout << "Hello";
```

You don't need to know:

- How characters are sent to the screen.
- How the operating system handles output.

`cout` is a high-level abstraction over the lower-level output operations.

---

## Why is it useful?

- Makes programs easier to write.
- Reduces complexity.
- Lets programmers focus on solving problems instead of low-level details.
- Makes code easier to read and maintain.

---

## Simple exam definition

> **High-level abstraction** is the process of hiding complex implementation details and providing a simple, easy-to-use interface for programmers.

**In one sentence:**
**High-level abstraction = "Use something without needing to know how it works internally."**

what is object oriented programming

**Object-Oriented Programming (OOP)** is a programming style where programs are built using **objects**, which contain **data (attributes)** and **functions (methods)** that operate on that data.

## Simple definition

> **Object-Oriented Programming (OOP)** is a programming paradigm that organizes code into **objects**, each representing a real-world entity with its own data and behavior.

---

## Real-life example

Think of a **Car**.

A car has:

- **Data (attributes):**
  - Color
  - Brand
  - Speed
- **Behavior (methods):**
  - Start()
  - Accelerate()
  - Brake()

In OOP, you can represent this as:

```
C++
class Car {
public:
    string brand;
    string color;

    void start() {
        cout << "Car started";
    }
};
```

Create an object:

```
C++
Car car1;

car1.brand = "Toyota";
car1.color = "Red";

car1.start();
```

Here:

- `Car` is a **class** (blueprint).
- `car1` is an **object** (a real car created from the blueprint).

---

## Why use OOP?

- ✅ Organizes code into reusable objects.
- ✅ Makes programs easier to understand and maintain.
- ✅ Reduces code duplication.
- ✅ Models real-world entities naturally.

---

## Four main principles of OOP

1. **Encapsulation** – Bundle data and methods together and control access.
2. **Abstraction** – Hide unnecessary implementation details.
3. **Inheritance** – Create new classes from existing classes.
4. **Polymorphism** – Allow the same interface to behave differently for different objects.

---

## Simple analogy

Imagine a **class** as a **blueprint for a house**.

- The blueprint describes what a house will have (rooms, doors, windows).
- Each actual house built from that blueprint is an **object**.

Similarly:

- **Class** = Blueprint
- **Object** = Real thing created from the blueprint

---

## Exam definition

> **Object-Oriented Programming (OOP)** is a programming paradigm that uses **classes and objects** to organize data and functions together, making programs more modular, reusable, and easier to maintain.

**In one sentence:**
**OOP = "A way of programming where data and the functions that work on it are grouped together into objects."**

> what is SOLID principle in CS

The **SOLID principles** are **five design principles in Object-Oriented Programming (OOP)** that help developers write code that is **easy to understand, maintain, and extend**.

Think of SOLID as **five rules for writing better object-oriented code**.

---

# S – Single Responsibility Principle (SRP)

**Rule:** A class should have **only one responsibility (one job).**

## ❌ Bad example

A `Student` class:

- Stores student data.
- Calculates grades.
- Prints reports.
- Saves data to a file.

This class has **too many responsibilities**.

## ✅ Good example

Separate them:

- `Student` → stores student data.
- `GradeCalculator` → calculates grades.
- `ReportPrinter` → prints reports.

Each class has **one job**.

---

# O – Open/Closed Principle (OCP)

**Rule:** Software should be **open for extension but closed for modification.**

This means:

- Add new features **without changing existing code**.

## Example

Suppose you have a `Shape` class.

Instead of changing it every time you add a new shape, create new classes:

- `Circle`
- `Rectangle`
- `Triangle`

The existing code stays the same.

---

# L – Liskov Substitution Principle (LSP)

**Rule:** A child class should be able to replace its parent class **without breaking the program**.

### Example

Parent:

```
Bird
```

Children:

```
Sparrow
Pigeon
```

Both can fly, so replacing `Bird` with `Sparrow` works correctly.

A common counterexample is making `Penguin` inherit from `Bird` if your code assumes every bird can fly. If a function expects a flying bird, substituting a penguin would break that expectation. In that design, the inheritance hierarchy should be reconsidered.

---

## I – Interface Segregation Principle (ISP)

**Rule:** Don't force a class to implement methods it doesn't need.

### ❌ Bad

Interface:

```
Machine
- print()
- scan()
- fax()
```

A simple printer doesn't need `scan()` or `fax()`.

### ✅ Good

Create smaller interfaces:

- `Printer`
- `Scanner`
- `Fax`

Each device implements only what it needs.

---

## D – Dependency Inversion Principle (DIP)

**Rule:** Depend on **abstractions (interfaces)**, not on concrete classes.

### ❌ Bad

A `Computer` class directly creates a `Keyboard`.

```
Computer → Keyboard
```

### ✅ Good

Depend on an abstraction:

```
Computer → InputDevice
                ↑
        Keyboard
        Mouse
```

Now you can replace the keyboard with another input device without changing the `Computer` class.

---

## Easy way to remember SOLID

| Letter | Principle | Simple Meaning |
|---|---|---|
| **S** | Single Responsibility | One class = one job |
| **O** | Open/Closed | Add new features without changing old code |
| **L** | Liskov Substitution | Child classes should work wherever the parent is expected |
| **I** | Interface Segregation | Don't force classes to implement unused methods |
| **D** | Dependency Inversion | Depend on interfaces/abstractions, not concrete implementations |

### Exam definition

> **SOLID** is a set of five object-oriented design principles that help developers create software that is maintainable, reusable, flexible, and easy to extend.

**In one sentence:**
**SOLID = Five principles for writing clean, maintainable, and extensible object-oriented code.**

> Entity-Relationship Model, where they actually used in CS

The **Entity-Relationship (ER) Model** is mainly used in **database design**. It helps you plan how data should be organized **before creating a database**.

## What is an ER Model?

An **ER Model** is a diagram that represents:

- **Entities** (things or objects)
- **Attributes** (properties of those things)

- **Relationships** (how the entities are connected)

### Example

For a college database:

- **Student**
  - Student_ID
  - Name
  - Age
- **Course**
  - Course_ID
  - Course_Name

Relationship:

- A **Student** enrolls in a **Course**.

This can be represented as:

```
Student ---- Enrolls ---- Course
```

## Where is it actually used?

### 1. Designing databases (Most common use)

Before creating tables in databases like:

- MySQL
- PostgreSQL
- Oracle
- SQL Server

developers first draw an ER diagram.

Example:

- Banking system
- Hospital management system
- School management system
- E-commerce websites

---

### 2. Software development

When building applications, developers use ER diagrams to decide:

- What data needs to be stored
- How different pieces of data relate to each other

Example:
An online shopping app has:

- Customer
- Product
- Order
- Payment

An ER model helps organize these relationships before coding.

---

### 3. College projects

Students often create ER diagrams for projects such as:

- Library Management System
- Student Management System
- Hotel Booking System
- Inventory Management System

---

### 4. Database administration

Database administrators (DBAs) use ER models to:

- Design new databases
- Improve existing databases
- Document database structure

---

## Real-world example: Amazon-like shopping website

Entities:

- Customer
- Product
- Order
- Payment

Relationships:

- Customer places Order.
- Order contains Product.

- Customer makes Payment.

This helps developers translate the design into database tables such as:

- `Customers`
- `Products`
- `Orders`
- `Payments`

---

## Why is the ER Model important?

- Makes database planning easier.
- Shows relationships clearly.
- Reduces design mistakes.
- Serves as a blueprint before creating database tables.
- Improves communication among developers and database designers.

---

## Exam definition

> The **Entity-Relationship (ER) Model** is a conceptual database design model used to represent entities, their attributes, and the relationships between them. It is primarily used to design and plan databases before implementing them in a Database Management System (DBMS).

**In one sentence:**
**The ER Model is a blueprint used to design a database before creating its tables and relationships.**

MVC (Model-View-Controller)

**MVC (Model–View–Controller)** is a **software architecture (design pattern)** used to organize an application into three separate parts. This makes the code easier to develop, test, and maintain.

## The three parts of MVC

### 1. Model

The **Model** manages the application's **data and business logic**.

Responsibilities:

- Stores data.
- Reads/writes data from the database.

- Performs business rules and calculations.

**Example:**
In a student management system, the `Student` model:

- Stores student information.
- Retrieves student records from the database.
- Updates student details.

---

### 2. View

The **View** is the **user interface (UI)**.

Responsibilities:

- Displays data to the user.
- Receives user input.
- Does not contain business logic.

**Examples:**

- A web page
- A login screen
- A student information page

---

### 3. Controller

The **Controller** acts as the **middleman** between the Model and the View.

Responsibilities:

- Receives user requests.
- Calls the Model to process data.
- Sends the results to the View.

---

## How MVC works

Suppose a user wants to view their profile.

```
User
  |
  ▼
Controller
  |
  ▼
Model ⟶ Database
  |
  ▼
Controller
```

```
  |
  ▼
View
  |
  ▼
User sees profile
```

### Step-by-step

1. The user clicks **"View Profile"**.
2. The **Controller** receives the request.
3. The **Controller** asks the **Model** for the user's data.
4. The **Model** gets the data from the database.
5. The **Model** returns the data to the **Controller**.
6. The **Controller** passes the data to the **View**.
7. The **View** displays the profile to the user.

---

## Real-world analogy: Restaurant

Imagine you're at a restaurant:

- **Model** = Kitchen (prepares the food and manages ingredients)
- **View** = Dining table (where the food is presented)
- **Controller** = Waiter (takes your order, communicates with the kitchen, and brings the food)

You don't go directly to the kitchen. The waiter (Controller) handles communication between you (View/User) and the kitchen (Model).

---

## Where is MVC used?

MVC is widely used in:

- Web applications
- Mobile applications
- Desktop applications

Many frameworks are based on MVC, including:

- ASP.NET MVC
- Spring MVC (Java)
- Ruby on Rails
- Laravel (PHP)

---

## Advantages

- Separates code into clear responsibilities.
- Easier to maintain and debug.
- Allows multiple developers to work on different parts simultaneously.
- Makes testing easier.
- Promotes code reuse.

---

## Exam definition

> **MVC (Model–View–Controller)** is a software architecture that divides an application into three components: **Model** (data and business logic), **View** (user interface), and **Controller** (handles user input and coordinates between the Model and View).

**In one sentence:**
**MVC separates an application into Model, View, and Controller so each part has a specific responsibility, making the application easier to build and maintain.**

what is Microservices with real world application

**Microservices** is a software architecture where a large application is divided into **many small, independent services**. Each service performs **one specific business function** and can be developed, deployed, and scaled independently.

## Simple definition

> **Microservices** is an architecture in which an application is built as a collection of small, independent services that communicate with each other.

---

## Real-world analogy

Imagine a **restaurant**.

Instead of one person doing everything:

- Taking orders
- Cooking
- Making drinks
- Collecting payment

Different people handle different tasks:

- 👨‍🍳 Chef → Cooks food
- 🥤 Drink maker → Prepares drinks
- 💵 Cashier → Handles payment
- 🤵 Waiter → Takes orders

Each person has **one responsibility**.

Microservices work the same way: each service focuses on one job.

---

# Example: Amazon

Amazon doesn't run as one huge program. Instead, it has many services.

```
Amazon Website
│
├── User Service
├── Product Service
├── Cart Service
├── Order Service
├── Payment Service
├── Inventory Service
└── Delivery Service
```

## What each service does

- **User Service** → Login, registration, user profile.
- **Product Service** → Product details and search.
- **Cart Service** → Shopping cart.
- **Order Service** → Places and tracks orders.
- **Payment Service** → Processes payments.
- **Inventory Service** → Checks stock.
- **Delivery Service** → Shipping and tracking.

When you place an order:

1. You click **Buy Now**.
2. **Order Service** receives the request.
3. It asks **Inventory Service** if the item is available.
4. It asks **Payment Service** to process the payment.
5. If payment succeeds, **Delivery Service** arranges shipping.

Each service communicates with the others, often using APIs.

---

# Another real-world application: Netflix

Netflix uses many microservices, such as:

- User authentication
- Movie catalog
- Recommendations
- Streaming
- Billing
- Viewing history

If the **recommendation service** has a problem, users can often still watch movies because the **streaming service** continues to work.

---

## Why use microservices?

- ✅ Easier to maintain because each service is small.
- ✅ Teams can work on different services independently.
- ✅ Services can be updated without redeploying the whole application.
- ✅ Individual services can be scaled based on demand (for example, scaling only the payment service during a big sale).
- ✅ A problem in one service is less likely to bring down the entire application if the system is designed to handle failures gracefully.

---

## Microservices vs Monolithic Architecture

| Monolithic | Microservices |
|---|---|
| One large application | Many small services |
| All code is together | Each service has its own responsibility |
| Harder to scale individual features | Individual services can be scaled independently |
| Updating one part often requires redeploying the whole application | Services can often be updated independently |

---

## Exam definition

> **Microservices** is a software architecture in which an application is divided into small, independent services. Each service performs a specific business function and communicates with other services through APIs or messaging.

### In one sentence

**Microservices = "Build one big application as many small, independent services, each responsible for one task."**

> what is API and its types details and there use cases

An **API (Application Programming Interface)** is a way for **two software applications to communicate with each other**.

Think of an API as a **messenger**:

- You (the client) send a request.
- The API delivers the request to another application (the server).
- The server processes it and sends back a response.

---

## Simple real-world example

Imagine you're at a restaurant.

- **You** → Customer
- **Waiter** → API
- **Kitchen** → Server

You don't go into the kitchen yourself. You tell the waiter what you want.

The waiter:

1. Takes your order.
2. Gives it to the kitchen.
3. Brings back your food.

The **waiter is the API**.

---

## Example in software

Suppose you're using a weather app.

When you open it:

```
Weather App
     │
     │ Request
     ▼
Weather API
     │
     ▼
Weather Server
     │
     │ Response
     ▼
Weather App
```

The weather app doesn't have weather information itself. It asks a weather API, which returns data such as:

- Temperature
- Humidity
- Wind speed

---

## Why are APIs used?

APIs allow applications to:

- Share data
- Use another application's features
- Avoid rebuilding existing functionality
- Connect different systems

Examples:

- Login with Google
- Pay with PayPal
- Google Maps inside Uber
- Weather data in mobile apps

---

## Types of APIs

There are several ways to classify APIs. Here are the most common ones.

## 1. REST API (Most Popular)

REST (Representational State Transfer) is the most widely used type for web applications.

Characteristics:

- Uses HTTP (GET, POST, PUT, DELETE)
- Usually exchanges data in JSON
- Simple and scalable

Example:

```
GET /users/1
```

Response:

```
JSON
```

```
{
  "id": 1,
  "name": "Alice"
}
```

### Use cases

- Mobile apps
- E-commerce websites
- Social media
- Banking applications

Examples:

- GitHub API
- Spotify API
- OpenWeather API

---

## 2. SOAP API

SOAP (Simple Object Access Protocol) is an older, highly standardized API protocol.

Characteristics:

- Uses XML
- Strict rules
- Built-in standards for security and reliability

Example use cases

- Banking systems
- Government systems
- Enterprise software

Example:

Banks often use SOAP because transactions require strict security and standardized message formats.

---

## 3. GraphQL API

GraphQL lets the client request exactly the data it needs.

Instead of:

```
Get User
```

You can request:

```
Name
Email
Phone
```

Only those fields are returned.

### Use cases

- Facebook
- Mobile apps
- Applications that need flexible data fetching

---

## 4. gRPC

gRPC is a high-performance API framework that uses Protocol Buffers instead of JSON.

Characteristics:

- Very fast
- Efficient communication
- Commonly used for communication between backend services

### Use cases

- Microservices
- Cloud platforms
- Real-time systems

---

# Types of APIs based on access

## 1. Public API

Available for anyone to use.

Examples:

- GitHub API
- Weather APIs
- Currency exchange APIs

Use case:
Developers build applications using public services.

---

## 2. Private API

Used only inside an organization.

Example:

A company's HR system communicates with its payroll system.

Use case:
Internal software integration.

---

## 3. Partner API

Shared only with approved business partners.

Example:

An airline allows travel agencies to access flight booking APIs.

Use case:
Business-to-business (B2B) integration.

---

## 4. Composite API

Combines multiple API calls into one request.

Example:

An online shopping app loads:

- User profile
- Shopping cart
- Recent orders

with a single API request.

Use case:
Reducing the number of requests and improving performance.

---

## Common HTTP methods in REST APIs

| Method | Purpose | Example |
|---|---|---|
| GET | Retrieve data | Get product details |
| POST | Create new data | Create a new user |
| PUT | Update an existing resource (often replacing it) | Update a user's profile |
| PATCH | Partially update a resource | Change only the user's email |
| DELETE | Remove data | Delete a product |

# Real-world applications of APIs

## 1. Google Maps

Uber uses the Google Maps API to:

- Show maps
- Calculate routes
- Estimate travel time

---

## 2. Payment APIs

Online stores use payment APIs to process:

- Credit cards
- UPI
- Digital wallets

Examples include services from Stripe, PayPal, and Razorpay.

---

## 3. Login APIs

Many websites let you:

- Login with Google
- Login with Facebook
- Login with Apple

These use authentication APIs instead of requiring a separate account.

---

## 4. Weather APIs

Weather apps retrieve:

- Temperature
- Forecast
- Wind speed

from weather service APIs.

---

## 5. Social Media APIs

Applications can:

- Post content
- Read comments
- Retrieve profile information (subject to permissions)

using APIs provided by social media platforms.

---

### 6. E-commerce APIs

An online shopping platform may have APIs for:

- Products
- Inventory
- Orders
- Payments
- Shipping

---

## Advantages of APIs

- Easy communication between applications.
- Reuse existing functionality instead of building everything from scratch.
- Save development time.
- Support integration between different platforms and services.
- Enable scalable and modular application design.

---

## Interview definition

> An **API (Application Programming Interface)** is a set of rules and protocols that allows different software applications to communicate and exchange data or functionality.

---

## Exam definition

> An **API (Application Programming Interface)** is a software interface that enables two or more applications to communicate by sending requests and receiving responses.

### Quick revision table

| API Type | Best For | Example Use Case |
|---|---|---|
| REST | General web and mobile apps | E-commerce, social media |
| SOAP | Secure enterprise systems | Banking, government |
| GraphQL | Flexible data retrieval | Facebook, mobile apps |

| API Type | Best For | Example Use Case |
|---|---|---|
| gRPC | High-performance service-to-service communication | Microservices, cloud systems |
| Public API | Open to developers | Weather, GitHub |
| Private API | Internal company systems | HR ↔ Payroll |
| Partner API | Business partners | Airline ↔ Travel agency |
| Composite API | Multiple operations in one request | Dashboard loading user, cart, and orders |

**In one sentence:**
**An API is a bridge that allows different software applications to communicate and share data or functionality in a standardized way.**

what is REST (Representational State Transfer)

**REST (Representational State Transfer)** is an **architectural style** for designing web APIs. It defines a set of principles for how clients (such as web or mobile apps) communicate with servers over HTTP.

In simple terms:

> **REST is a standard way for applications to communicate over the internet using HTTP requests.**

---

## Simple real-world example

Imagine a **library**.

- You ask the librarian for a book.
- The librarian finds the book.
- The librarian gives it to you.

Here:

- **You** → Client
- **Librarian** → REST API
- **Library** → Server/Database

You request information, and the server returns it.

---

## How REST works

Suppose you open a shopping app to view a product.

```
Mobile App
     │
     │ GET /products/101
     ▼
```

```
REST API
    │
    ▼
Database
    │
    │ Product details
    ▼
REST API
    │
    ▼
Mobile App
```

The app sends an HTTP request to the REST API, which retrieves the data and returns it.

---

## HTTP methods used in REST

REST commonly uses these HTTP methods:

| Method | Purpose | Example |
|---|---|---|
| **GET** | Retrieve data | Get product details |
| **POST** | Create new data | Create a new user |
| **PUT** | Replace an existing resource | Update a user's profile |
| **PATCH** | Partially update a resource | Change only the user's phone number |
| **DELETE** | Delete data | Delete a product |

---

## Example 1: GET

Request:

```http
GET /students/1
```

Response:

```JSON
{
  "id": 1,
  "name": "Rahul",
  "age": 20
}
```

This retrieves information about student **1**.

---

## Example 2: POST

Create a new student:

```http
```

```
POST /students
```

Body:

```
JSON
{
  "name": "Anita",
  "age": 21
}
```

The server creates the new student.

---

## Example 3: DELETE

```
http
DELETE /students/1
```

This removes student **1**.

---

## Why is it called "Representational State Transfer"?

The name can be understood by breaking it into parts:

- **Representation**: Data is sent in a format such as JSON or XML.
- **State**: The current information about a resource (for example, a user's profile).
- **Transfer**: That representation is transferred between the client and the server.

For example, when you request:

```
http
GET /products/10
```

The server sends a **representation** of product **10**, usually as JSON.

---

## What is a Resource?

In REST, everything is treated as a **resource**.

Examples:

```
/students
/products
/orders
/users
/books
```

Each resource is identified by a URL.

Examples:

```
GET /users/5
GET /products/10
DELETE /orders/22
```

## Why do developers use REST?

- Simple to understand.
- Works with standard HTTP.
- Supported by almost every programming language.
- Scalable for web and mobile applications.
- JSON responses are compact and easy to parse.

---

## Real-world applications

REST APIs are widely used in:

- E-commerce websites
- Banking applications
- Social media platforms
- Food delivery apps
- Ride-sharing apps
- Weather applications

Examples:

- An online store fetching product details.
- A banking app retrieving account balances.
- A food delivery app placing orders.

---

## Advantages of REST

- Easy to learn and implement.
- Platform independent.
- Supports many data formats (JSON is the most common).
- Scales well for large applications.
- Stateless requests simplify server design.

---

## What does "stateless" mean?

REST is **stateless**, which means:

> **Each request contains all the information the server needs to process it. The server does not rely on information from previous requests.**

For example:

```
GET /profile
Authorization: Bearer <token>
```

If you make the same request again, you send the required authentication information again. The server processes each request independently.

---

# Exam definition

> **REST (Representational State Transfer)** is an architectural style for designing web APIs that uses HTTP methods to access and manipulate resources. It is stateless, scalable, and commonly exchanges data in JSON format.

# Quick revision

| Concept | Meaning |
|---|---|
| REST | Architecture for building web APIs |
| Resource | An object such as a user, product, or order |
| URL | Identifies a resource |
| GET | Read data |
| POST | Create data |
| PUT | Replace data |
| PATCH | Partially update data |
| DELETE | Remove data |
| JSON | Most common format for exchanging data |
| Stateless | Every request is independent and contains the information needed to process it |

**In one sentence:**
**REST is a standard way for applications to communicate over HTTP by performing operations on resources using methods like GET, POST, PUT, PATCH, and DELETE.**

what is Data Normalization

# Data Normalization

**Data Normalization** is a **database design technique** used to organize data into tables in a way that **reduces duplication (redundancy)** and **improves data consistency**.

In simple words:

> **Normalization means dividing large tables into smaller related tables and connecting them using relationships so that data is stored efficiently.**

---

## Real-world example (without normalization)

Suppose a college stores student and course information in one table:

| Student_ID | Student_Name | Course | Teacher |
|---|---|---|---|
| 101 | Rahul | Database | Sharma |
| 102 | Priya | Database | Sharma |
| 103 | Amit | Programming | Verma |

Problem:

- "Database" and "Sharma" are repeated multiple times.
- If the teacher name changes, we must update many rows.
- Duplicate data wastes storage.

---

## After normalization

Split the data into separate tables:

### Student Table

| Student_ID | Student_Name |
|---|---|
| 101 | Rahul |
| 102 | Priya |
| 103 | Amit |

### Course Table

| Course_ID | Course_Name | Teacher |
|---|---|---|
| C1 | Database | Sharma |
| C2 | Programming | Verma |

### Enrollment Table

| Student_ID | Course_ID |
|---|---|
| 101 | C1 |

| Student_ID | Course_ID |
|---|---|
| 102 | C1 |
| 103 | C2 |

Now:

- No repeated course information.
- Updates are easier.
- Data is more organized.

---

# Why do we use normalization?

## 1. Reduce data redundancy

Avoid storing the same information multiple times.

Example:

❌ Before:

```
Rahul - Computer Science - Dr. Sharma
Priya - Computer Science - Dr. Sharma
Amit  - Computer Science - Dr. Sharma
```

The department and teacher repeat.

✅ After:

```
Student Table
Department Table
Teacher Table
```

Store information once.

---

## 2. Improve data consistency

If a teacher's name changes:

Without normalization:

- Update many rows.
- Risk of missing some.

With normalization:

- Update one place.

---

## 3. Avoid database anomalies

Normalization prevents three common problems:

### A) Insert Anomaly

Problem:
You cannot add some information without adding unrelated data.

Example:
You cannot add a new course until at least one student enrolls.

---

### B) Update Anomaly

Problem:
The same information exists in multiple places and must be updated everywhere.

Example:
Changing a teacher's name requires updating many rows.

---

### C) Delete Anomaly

Problem:
Deleting one record accidentally removes important information.

Example:
Deleting the last student enrolled in a course also deletes course details.

---

## Normal Forms in Database

Normalization is performed in stages called **normal forms**.

---

## 1NF (First Normal Form)

Rules:

- Each column contains only atomic (single) values.
- No repeating groups.

### ❌ Not 1NF

| Student | Phone Numbers |
|---|---|
| Rahul | 9876, 8765 |

Multiple phone numbers in one field.

### ✅ 1NF

| Student | Phone |
|---|---|
| Rahul | 9876 |

| Student | Phone |
|---|---|
| Rahul | 8765 |

## 2NF (Second Normal Form)

Rules:

- Must already be in 1NF.
- No partial dependency on part of a composite key.

Used mainly when a table has a combined primary key.

## 3NF (Third Normal Form)

Rules:

- Must already be in 2NF.
- No dependency between non-key attributes.

Example:

❌

| Student_ID | Department_ID | Department_Name |
|---|---|---|

**`Department_Name`** depends on **`Department_ID`**, not directly on Student_ID.

Separate department data into another table.

## BCNF (Boyce-Codd Normal Form)

A stronger version of 3NF.

It handles some complex dependency problems that 3NF cannot solve.

## Where is normalization used?

Normalization is used in almost every large database system:

### Banking systems

- Customer information
- Accounts

- Transactions

### E-commerce websites

- Users
- Products
- Orders
- Payments

### Hospital systems

- Patients
- Doctors
- Appointments

### University systems

- Students
- Courses
- Grades

---

## Normalization vs Denormalization

| Normalization | Denormalization |
|---|---|
| Reduces duplicate data | Adds some duplication |
| Improves consistency | Improves query speed |
| Uses more tables | Uses fewer tables |
| Common in transactional systems | Common in analytics systems |

---

## Exam definition

> **Data Normalization is a database design process that organizes data into multiple related tables to reduce redundancy, eliminate anomalies, and improve data integrity.**

### In one sentence:

**Normalization = "Organize database data properly so that the same information is not stored repeatedly."**

complete details on ACID Properties

# ACID Properties in Database Systems

**ACID properties** are a set of four important rules that ensure **database transactions are reliable, accurate, and safe**.

ACID stands for:

- **A** → Atomicity
- **C** → Consistency
- **I** → Isolation
- **D** → Durability

They are mainly used in **Database Management Systems (DBMS)** to handle transactions safely.

---

## What is a Transaction?

A **transaction** is a sequence of database operations that performs one complete task.

Example: **Bank money transfer**

Transfer ₹1000 from Account A to Account B:

1. Deduct ₹1000 from A.
2. Add ₹1000 to B.

These two operations together form **one transaction**.

```
Account A (-₹1000)
        +
Account B (+₹1000)
        =
Complete Transaction
```

If something fails during the process, the database must handle it correctly. This is where ACID properties help.

---

## 1. Atomicity (All or Nothing)

### Meaning:

> A transaction is treated as a single unit. Either all operations complete successfully, or none of them happen.

"Atomic" means **indivisible**.

### Example:

Money transfer:

```
Step 1: Remove ₹1000 from Account A ✓
Step 2: Add ₹1000 to Account B ✗ (failed)
```

Without atomicity:

- Money disappears from A.
- B does not receive money.

With atomicity:

- The database cancels the entire transaction.
- Money remains in A.

This is called **rollback**.

---

### Real-world examples:

- Online payment
- ATM withdrawal
- Booking tickets

If payment fails, the booking should not be completed.

---

## 2. Consistency (Maintains Correct Data)

### Meaning:

> A transaction must take the database from one valid state to another valid state.

The database rules and constraints must always remain correct.

---

### Example:

Bank account rule:

```
Balance cannot be negative
```

Before transaction:

```
Account A = ₹5000
Account B = ₹3000
```

Transfer:

```
A sends ₹1000 to B
```

After transaction:

```
Account A = ₹4000
Account B = ₹4000
```

The total money remains:

```
₹8000 before
₹8000 after
```

The database stays consistent.

---

### Consistency maintains:

- Primary key rules
- Foreign key rules
- Data types
- Business rules
- Constraints

---

## 3. Isolation (Transactions Don't Interfere)

### Meaning:

> Multiple transactions happening at the same time should not affect each other.

Each transaction should behave as if it is running alone.

---

## Example:

Suppose a bank account has:

```
Balance = ₹10,000
```

Two users try to withdraw ₹7000 simultaneously.

Without isolation:

Transaction A:

```
Reads balance: ₹10,000
Withdraws ₹7000
```

Transaction B:

```
Reads balance: ₹10,000
Withdraws ₹7000
```

The system may incorrectly allow both.

Result:

```
Balance = -₹4000
```

Wrong!

---

With isolation:

Transaction A completes first:

```
₹10,000 - ₹7,000 = ₹3,000
```

Transaction B sees the updated balance and fails.

---

### Isolation prevents:

- Dirty reads
- Lost updates
- Incorrect intermediate data

---

## 4. Durability (Permanent Changes)

### Meaning:

> Once a transaction is successfully completed, the changes are permanently stored even if the system crashes.

---

## Example:

You buy a product online.

Process:

```
Payment successful ✓
Order created ✓
```

Suddenly:

```
Server crashes
```

After restart:

```
Order still exists
Payment record still exists
```

Because the database saved the transaction permanently.

---

Durability is achieved using:

- Transaction logs
- Backup systems
- Recovery mechanisms

---

## ACID Properties Summary

| Property | Meaning | Protects Against |
|---|---|---|
| **Atomicity** | All operations happen or none happen | Partial transactions |
| **Consistency** | Database remains valid | Incorrect data |
| **Isolation** | Transactions don't interfere | Concurrent access problems |
| **Durability** | Completed changes are permanent | Data loss after crashes |

---

## ACID Example: ATM Withdrawal

Suppose you withdraw ₹5000.

### Atomicity

Either:

- Money is deducted and cash is given.

OR

- Nothing happens.

---

### Consistency

Account balance rules remain valid.

Example:

```
Before: ₹20,000
After: ₹15,000
```

### Isolation

Another ATM cannot interfere while your withdrawal is processing.

---

### Durability

After successful withdrawal:

```
New balance = ₹15,000
```

The database remembers it permanently.

---

## ACID in SQL Databases

Most relational databases support ACID transactions:

Examples:

- MySQL (InnoDB)
- PostgreSQL
- Oracle Database
- Microsoft SQL Server

Example SQL transaction:

```sql
SQL
START TRANSACTION;

UPDATE account
SET balance = balance - 1000
WHERE id = 1;

UPDATE account
SET balance = balance + 1000
WHERE id = 2;

COMMIT;
```

If something fails:

```sql
SQL
ROLLBACK;
```

## ACID vs BASE

Some distributed databases use **BASE** instead of strict ACID.

| ACID | BASE |
|---|---|
| Strong consistency | Eventual consistency |
| Used in banking systems | Used in large distributed systems |
| More strict | More flexible |
| Example: Banking | Example: Social media feeds |

---

## Where ACID is used?

### Banking

- Money transfers
- Account updates

### E-commerce

- Orders
- Payments
- Inventory

### Airline systems

- Ticket booking

### Healthcare

- Patient records

### Stock trading

- Buying and selling shares

---

## Interview Definition

> **ACID properties are a set of database transaction properties that ensure reliability and correctness. They guarantee that transactions are Atomic, Consistent, Isolated, and Durable.**

---

## Easy way to remember

**A → All or nothing**
**C → Correct data always**
**I → Independent transactions**
**D → Data stays forever**

**In one sentence:**
**ACID ensures that database transactions happen safely, correctly, and permanently even when errors or failures occur.**

explain about CAP Theorem

## CAP Theorem in Distributed Systems

**CAP Theorem** is a fundamental concept in **distributed database systems**. It explains the trade-off between three important properties:

- **C → Consistency**
- **A → Availability**
- **P → Partition Tolerance**

The theorem states:

> **A distributed system can guarantee at most two out of these three properties at the same time.**

This is also called the **CAP trade-off**.

---

## Why do we need CAP Theorem?

Modern applications often run on **multiple servers in different locations**.

Example:

A social media application may have:

```
        User
         |
   -------------
   |           |
Server A     Server B
(New York)   (London)
```

Both servers store copies of the same data.

The challenge is:

- What happens if the servers cannot communicate?
- Should the system show old data?
- Should it stop accepting requests?

CAP Theorem helps make these decisions.

---

## The Three Properties

## 1. Consistency (C)

### Meaning:

> Every user sees the same, latest version of data at the same time.

After a successful write, every read gets the updated value.

### Example:

A bank account balance:

```
Account balance = ₹5000
```

You deposit ₹1000.

After the transaction:

```
Every server shows ₹6000
```

No server shows the old value.

### Used in:

- Banking systems
- Payment systems
- Stock trading

---

## 2. Availability (A)

### Meaning:

> The system always responds to requests, even if some servers have problems.

The user gets a response, even if it may not be the latest data.

### Example:

A shopping website during a server failure:

```
User requests product information
            ↓
Server responds with available data
```

The website continues working.

### Used in:

- Social media
- Search engines
- Content platforms

---

## 3. Partition Tolerance (P)

### Meaning:

> The system continues working even when communication between servers is broken.

A **network partition** happens when servers cannot communicate with each other.

Example:

```
Server A ❌ Server B

(Network connection lost)
```

Both servers continue operating independently.

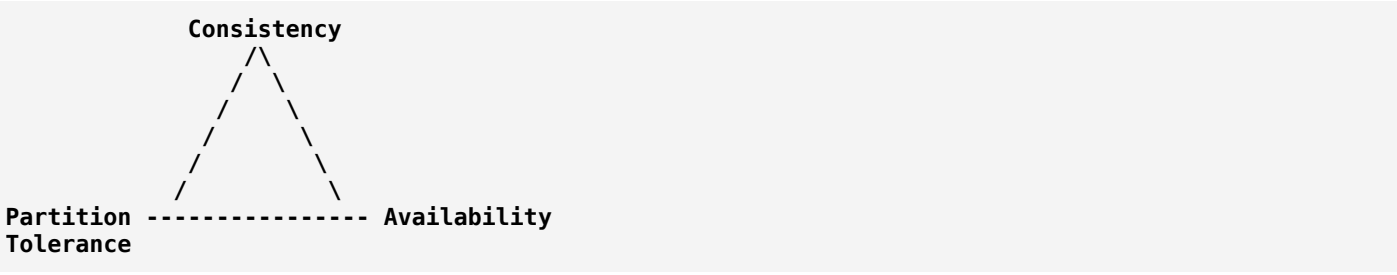
### Used in:

- Cloud systems
- Distributed databases
- Global applications

---

## The CAP Triangle

A distributed system chooses two:




You cannot perfectly achieve all three at the same time during a network failure.

---

## CAP Combinations

## 1. CP System

(Consistency + Partition Tolerance)

A CP system chooses correct data over availability.

During a network problem:

- Some requests may fail.
- But data remains correct.

Example:

```
Network failure occurs

System says:
"Please try again later"
```

### Used in:

- Banking systems

- Distributed databases requiring accuracy

Examples:

- MongoDB (when configured for strong consistency)
- HBase

---

## 2. AP System

(Availability + Partition Tolerance)

An AP system chooses availability over immediate consistency.

During a network problem:

- The system continues responding.
- Data may temporarily differ.

Later, data becomes synchronized.

This is called **eventual consistency**.

Example:

Social media likes:

```
User A sees:
100 likes

User B sees:
98 likes

Later:
Both see 100 likes
```

### Used in:

- Social networks
- Content delivery systems

Examples:

- Cassandra
- Amazon Dynamo-style systems

---

## 3. CA System

(Consistency + Availability)

A CA system provides:

- Correct data

- Always available responses

But it assumes there is **no network partition**.

In real distributed systems, network failures are unavoidable, so pure CA systems are mostly possible only in single-node or tightly connected systems.

Examples:

- Traditional single-server relational databases

---

## Real-world Example: Online Shopping

Imagine an e-commerce system.

## During normal operation:

```
Product stock = 10
```

A customer buys one:

```
Stock = 9
```

All servers update.

---

## During network failure:

Server A:

```
Stock = 9
```

Server B:

```
Stock = 10
```

The system must choose:

### Option 1: Consistency

Stop sales until servers agree.

(CP)

### Option 2: Availability

Continue selling and fix stock later.

(AP)

---

## CAP Theorem vs ACID

| CAP | ACID |
|---|---|
| Deals with distributed systems | Deals with database transactions |
| Focuses on network failures | Focuses on transaction correctness |
| Consistency, Availability, Partition tolerance | Atomicity, Consistency, Isolation, Durability |
| Used in distributed databases | Used mainly in relational databases |

## CAP Theorem in Popular Databases

| Database | Common CAP Choice |
|---|---|
| PostgreSQL | CA (single-node), CP in distributed setups |
| MongoDB | CP (strong consistency configurations) |
| Cassandra | AP |
| DynamoDB | AP/CP depending on configuration |
| HBase | CP |

## Important Point

Many people misunderstand CAP as:

❌ "A database can only ever have two properties."

Correct meaning:

✅ **When a network partition occurs, you must choose between consistency and availability.**

Partition tolerance is usually required in distributed systems because network failures can happen.

## Exam Definition

> **CAP Theorem states that a distributed computer system cannot simultaneously guarantee Consistency, Availability, and Partition Tolerance during a network partition. It can provide only two of these three guarantees at the same time.**

## Easy way to remember:

**C = Correct data**
**A = Always available**
**P = Problems in network**

During a network problem:

- Choose **Correct data** → CP
- Choose **Always available** → AP

**In one sentence:**
**CAP Theorem explains the trade-off between keeping data perfectly consistent and keeping a distributed system available when network failures occur.**

> explain DRY (Don't Repeat Yourself)

# DRY (Don't Repeat Yourself) Principle

**DRY (Don't Repeat Yourself)** is a software development principle that says:

> **Every piece of knowledge or logic in a system should have a single, clear representation.**

In simple words:

**Do not write the same code, logic, or information multiple times. Write it once and reuse it.**

---

# Why was DRY introduced?

When the same code is repeated in many places:

- Fixing bugs becomes harder.
- Changes require editing many files.
- Code becomes longer and confusing.
- Errors are more likely.

DRY helps make software:

- Easier to maintain.
- Easier to update.
- More reusable.

---

# Example: Without DRY

Suppose you calculate a discount in multiple places.

### Order page:

```
C++
double finalPrice = price - (price * 0.10);
```

### Invoice page:

```C++
double finalPrice = price - (price * 0.10);
```

### Payment page:

```C++
double finalPrice = price - (price * 0.10);
```

The same logic is repeated three times.

---

## Problem

Later, the discount changes from **10% to 15%**.

You must find and update every location:

```C++
price * 0.15
```

If you miss one place, your application gives incorrect results.

---

## Applying DRY

Create a reusable function:

```C++
double calculateDiscount(double price) {
    return price - (price * 0.10);
}
```

Now use it everywhere:

```C++
double orderAmount = calculateDiscount(1000);
double invoiceAmount = calculateDiscount(2000);
double paymentAmount = calculateDiscount(500);
```

Now the discount logic exists in **one place**.

---

## Real-world example

Imagine a company stores customer information.

### Without DRY:

Customer address is stored separately in:

- Order system

- Billing system
- Delivery system

If the customer changes address, all three places need updating.

### With DRY:

A single customer profile service stores the address.

Other systems use that information.

---

## DRY in different areas

## 1. Code reuse

Instead of:

```
C++
cout << "Welcome User";
cout << "Welcome Admin";
cout << "Welcome Guest";
```

Create:

```
C++
void welcome(string name) {
    cout << "Welcome " << name;
}
```

## 2. Avoid repeated database logic

Instead of writing the same SQL query everywhere:

```
SQL
SELECT * FROM users WHERE id = 10;
```

Create a reusable database function or service.

---

## 3. Avoid repeated UI components

In web development:

Instead of creating multiple copies of a button design:

```
Login Button
Register Button
Submit Button
```

Create one reusable button component.

## DRY vs WET

WET is often jokingly expanded as:

> **Write Everything Twice**

or

> **We Enjoy Typing**

| DRY | WET |
|---|---|
| Code written once | Code repeated |
| Easier maintenance | Harder maintenance |
| Less duplication | More duplication |
| Fewer bugs | More chances of errors |

## DRY does NOT mean "never repeat code"

A common misunderstanding:

❌ "Never write the same line twice."

That is not the goal.

The goal is:

✅ Avoid repeating **business logic, rules, and important knowledge**.

Sometimes small duplication is better than creating unnecessary complex abstractions.

Example:

Two simple print statements may not need a function, but a repeated payment calculation definitely should.

## DRY with OOP

DRY is supported by OOP concepts:

### Inheritance

Reuse existing class functionality:

```
C++
class Vehicle {
public:
    void start();
};
```

```
class Car : public Vehicle {
};
```

### Functions

Reuse common operations:

```
C++
calculateTax();
```

### Classes

Create reusable objects:

```
C++
class User {
};
```

## DRY and Software Design

DRY works well with other principles:

| Principle | Relationship |
| --- | --- |
| DRY | Avoid duplication |
| SOLID | Improve object-oriented design |
| KISS | Keep solutions simple |
| YAGNI | Don't build unnecessary features |

---

## Real-world applications

DRY is used in:

### Web development

- Reusable components
- Shared layouts
- Common API functions

### Backend systems

- Common business logic
- Shared services

### Mobile apps

- Reusable UI components

### Databases

- Avoiding duplicate data structures

---

## Interview Definition

> **DRY (Don't Repeat Yourself) is a software development principle that encourages developers to avoid duplication by keeping each piece of logic or knowledge in a single, reusable place.**

---

## Easy way to remember:

**Bad approach:**

> "Copy the code wherever I need it."

**DRY approach:**

> "Create it once and reuse it."

**In one sentence:**
**DRY = Write code once, reuse it everywhere, and keep one source of truth.**

explain KISS (Keep It Simple, Stupid)

## KISS (Keep It Simple, Stupid) Principle

**KISS (Keep It Simple, Stupid)** is a software design principle that says:

> **A system should be designed as simply as possible, avoiding unnecessary complexity.**

In simple words:

**Choose the simplest solution that solves the problem.**

The word "Stupid" is not meant as an insult; it is a traditional phrase used to remind designers to avoid making solutions unnecessarily complicated.

---

## Why is KISS important?

Complex code creates problems:

- Harder to understand.
- Difficult to maintain.
- More bugs.

- Takes longer to modify.
- New developers struggle to work with it.

Simple code is usually:

- Easier to read.
- Easier to test.
- Easier to fix.
- Easier to improve.

---

## Real-world analogy

Imagine you need to open a door.

### Simple solution:

Use a key:

```
Key → Unlock door
```

### Overcomplicated solution:

Create:

- Fingerprint scanner
- Face recognition
- Voice authentication
- AI verification

for a normal bedroom door.

The extra complexity does not provide enough value.

KISS says: **use the simplest solution that works.**

---

## Programming example

## Without KISS

A programmer creates a complex function to check if a number is even:

```
C++
bool isEven(int number) {
    vector<int> values;
    values.push_back(number);

    if (values[0] % 2 == 0) {
```

```
        return true;
    }
    return false;
}
```

This works, but it is unnecessarily complicated.

---

## Applying KISS

Simple version:

```C++
bool isEven(int number) {
    return number % 2 == 0;
}
```

Same result, less complexity.

---

## Another example: User login

## Complex design

A small website creates:

```
Login Manager
       |
Authentication Factory
       |
Security Provider
       |
User Verification Engine
       |
Database Connector
```

For a simple application, this may be unnecessary.

---

## KISS design

```
Login Function
       |
   Database
```

Start simple. Add complexity only when needed.

---

## KISS in software development

## 1. Simple code structure

Prefer:

```
C++
total = price + tax;
```

over creating unnecessary classes for a simple calculation.

---

## 2. Simple algorithms

If searching a small list:

Use:

```
Linear Search
```

instead of building a complicated search system.

---

## 3. Simple architecture

A small application may not need:

- Multiple servers
- Complex microservices
- Advanced design patterns

A simple monolithic design may be better.

---

## KISS and other principles

| Principle | Meaning |
|---|---|
| **KISS** | Keep solutions simple |
| **DRY** | Avoid repeating code |
| **YAGNI** | Don't add features you don't need yet |
| **SOLID** | Create maintainable object-oriented designs |

---

## KISS vs Overengineering

**Overengineering** means creating a solution that is much more complex than the actual problem requires.

Example:

Problem:

> Store a user's name.

Simple:

```
User table → Name field
```

Overengineered:

```
User Service
   ↓
Name Processing Engine
   ↓
Metadata Pipeline
   ↓
AI Name Optimization Service
```

The second solution adds complexity without benefit.

---

## Benefits of KISS

✅ Easier code review
✅ Faster development
✅ Fewer bugs
✅ Easier debugging
✅ Easier maintenance
✅ Better teamwork

---

## When NOT to use extreme simplicity

KISS does **not** mean ignoring future needs.

For example:

- A banking system needs security.
- A medical system needs reliability.
- A large application needs proper architecture.

The goal is:

> **Keep the design as simple as possible, but not simpler than the requirements allow.**

---

## Interview Definition

> **KISS (Keep It Simple, Stupid) is a software engineering principle that encourages developers to avoid unnecessary complexity and create simple, understandable, and maintainable solutions.**

---

## Easy way to remember:

**Before KISS:**

> "How can I make this more advanced?"

**After KISS:**

> "What is the simplest way to solve this correctly?"

**In one sentence:**
**KISS = Simple solutions are usually better than unnecessarily complicated ones.**

explain detailed way in CRUD (Create, Read, Update, Delete)

## CRUD (Create, Read, Update, Delete) — Detailed Explanation

**CRUD** is a fundamental concept in software development and database systems. It represents the **four basic operations that can be performed on data** stored in a database.

Almost every application that stores data uses CRUD operations.

Examples:

- Social media apps (posts, comments, profiles)
- E-commerce apps (products, orders, customers)
- Banking systems (accounts, transactions)
- Student management systems (student records)

CRUD stands for:

| Letter | Operation | Meaning |
|---|---|---|
| **C** | Create | Add new data |
| **R** | Read | Retrieve existing data |
| **U** | Update | Modify existing data |
| **D** | Delete | Remove data |

---

## Real-world Example: Student Management System

Imagine a database table called **Students**:

| Student_ID | Name | Age | Course |
|---|---|---|---|
| 101 | Rahul | 20 | Computer Science |
| 102 | Priya | 21 | Mathematics |

Now let's see CRUD operations.

# 1. CREATE (Adding New Data)

## Meaning

**Create** means inserting new records into the database.

Example:

A new student joins the college:

```
Name: Amit
Age: 22
Course: Physics
```

The system creates a new record:

| Student_ID | Name | Age | Course |
|---|---|---|---|
| 103 | Amit | 22 | Physics |

## SQL Example

```
SQL
INSERT INTO Students
(Student_ID, Name, Age, Course)
VALUES
(103, 'Amit', 22, 'Physics');
```

## Real-world examples

### Instagram

Creating a new post:

```
User uploads photo
        ↓
New post record created
```

### E-commerce

Adding a new product:

```
Product name
Price
Category
Stock
```

are stored in the database.

# 2. READ (Retrieving Data)

## Meaning

**Read** means fetching existing data from a database.

Examples:

- Viewing your profile.
- Searching for a product.
- Checking bank balance.

---

## SQL Example

Get all students:

```
SQL
SELECT * FROM Students;
```

Result:

| Student_ID | Name | Age | Course |
|---|---|---|---|
| 101 | Rahul | 20 | Computer Science |
| 102 | Priya | 21 | Mathematics |

---

Get a specific student:

```
SQL
SELECT *
FROM Students
WHERE Student_ID = 101;
```

Output:

```
Rahul, 20, Computer Science
```

## Real-world examples

### YouTube

When you open YouTube:

```
Request videos
      ↓
Database returns videos
```

```
      ↓
App displays videos
```

### Banking

When you check your account:

```
Bank App
   ↓
Account Database
   ↓
Balance Information
```

## 3. UPDATE (Modifying Existing Data)

## Meaning

**Update** means changing existing information.

Example:

A student changes course:

Before:

| Student_ID | Name | Course |
|---|---|---|
| 101 | Rahul | Computer Science |

After:

| Student_ID | Name | Course |
|---|---|---|
| 101 | Rahul | Data Science |

---

## SQL Example

```
SQL
UPDATE Students
SET Course = 'Data Science'
WHERE Student_ID = 101;
```

## Real-world examples

### User profile update

Before:

```
Name: Rahul
City: Delhi
```

User changes city:

```
Name: Rahul
City: Mumbai
```

### Online shopping

Product stock update:

Before:

```
Laptop Stock = 50
```

After selling products:

```
Laptop Stock = 49
```

## 4. DELETE (Removing Data)

## Meaning

**Delete** means removing records from a database.

Example:

A student leaves the college.

Remove:

```
Student_ID = 103
```

## SQL Example

```SQL
DELETE FROM Students
WHERE Student_ID = 103;
```

## Real-world examples

### Social media

Deleting a post:

```
User clicks Delete
        ↓
Post removed from database
```

### E-commerce

Removing a discontinued product:

```
Product removed
```

## CRUD in Web Applications

A typical web application works like this:

```
          User Interface
               |
               |
          Controller/API
               |
               |
          Database
```

Example: Online shopping website

---

## CREATE

User clicks:

```
Add New Product
```

Flow:

```
Admin Panel
     |
     ↓
Product API
     |
     ↓
Database INSERT
```

## READ

User searches:

```
"Mobile phone"
```

Flow:

```
Search Page
     |
     ↓
Product API
     |
     ↓
Database SELECT
     |
     ↓
Display Products
```

## UPDATE

Admin changes price:

```
Old Price: ₹20,000
New Price: ₹18,000
```

Flow:

```
Admin Panel
     |
     ↓
Update API
     |
     ↓
Database UPDATE
```

# DELETE

Admin removes product:

```
Delete Product
```

Flow:

```
Admin Panel
     |
     ↓
Delete API
     |
     ↓
Database DELETE
```

# CRUD with REST API

CRUD operations map directly to HTTP methods:

| CRUD | HTTP Method | Example |
|---|---|---|
| Create | POST | Create a user |
| Read | GET | Get user details |
| Update | PUT/PATCH | Update user profile |
| Delete | DELETE | Delete user |

Example API:

## Create User

Request:

```
POST /users
```

Data:

```
JSON
```

```
{
 "name": "Rahul",
 "age": 20
}
```

### Read User

```
GET /users/101
```

Response:

```
JSON
{
 "id":101,
 "name":"Rahul"
}
```

### Update User

```
PUT /users/101
```

Data:

```
JSON
{
 "name":"Amit"
}
```

### Delete User

```
DELETE /users/101
```

## CRUD in Different Technologies

## SQL Database

| Operation | SQL Command |
|---|---|
| Create | INSERT |
| Read | SELECT |
| Update | UPDATE |
| Delete | DELETE |

## MongoDB

| CRUD | MongoDB Command |
|---|---|
| Create | insertOne() |

| CRUD | MongoDB Command |
|---|---|
| Read | find() |
| Update | updateOne() |
| Delete | deleteOne() |

## Object-Oriented Programming

Example:

```
C++
class Student {
public:
    void createStudent();
    void readStudent();
    void updateStudent();
    void deleteStudent();
};
```

## CRUD vs Database Transactions

CRUD describes **what operation happens**.

A transaction describes **how multiple operations are handled safely**.

Example:

Bank transfer:

```
UPDATE Account A
UPDATE Account B
```

These two CRUD updates should happen inside one transaction using ACID properties.

## CRUD Security Considerations

Real applications must control CRUD operations.

Example:

A normal user may:

✅ Read profile
✅ Update own profile

But cannot:

❌ Delete another user's account
❌ Update another user's data

This is handled using:

- Authentication

- Authorization
- Access control

## CRUD in Different Applications

| Application | Create | Read | Update | Delete |
|---|---|---|---|---|
| Instagram | Upload post | View posts | Edit caption | Delete post |
| Amazon | Add product/order | View products | Change order details | Remove product |
| Banking | Create account | Check balance | Update details | Close account |
| College System | Add student | View records | Edit information | Remove student |

## Interview Definition

> **CRUD is a set of four basic database operations: Create, Read, Update, and Delete. These operations represent the fundamental ways applications interact with stored data.**

## Easy way to remember

**C → Create something new**
**R → Retrieve something existing**
**U → Change something existing**
**D → Remove something existing**

**In one sentence:**
**CRUD is the foundation of data management: every application that stores information needs the ability to create, read, update, and delete data.**

DDD (Domain-Driven Design)

## DDD (Domain-Driven Design)

**Domain-Driven Design (DDD)** is a software design approach that focuses on building software based on the **business domain (the real-world problem area)** rather than focusing only on technical details.

In simple words:

> **DDD means designing software by deeply understanding the business problem and organizing the code around business concepts.**

## Why do we need DDD?

In large software systems, the biggest challenge is usually not writing code — it is understanding **complex business rules**.

Examples of complex domains:

- Banking
- Healthcare
- Insurance
- E-commerce
- Logistics
- Airline systems

A developer should first understand:

- What does the business do?
- What are the important concepts?
- What rules must be followed?

Then design the software around those concepts.

---

## Real-world example: Banking System

A bank has concepts like:

- Customer
- Account
- Transaction
- Loan
- Interest
- Payment

A normal programming approach might organize code around technical layers:

```
Controllers
    |
Services
    |
Database
```

DDD organizes around business concepts:

```
Banking Domain

├── Customer
├── Account
├── Transaction
├── Loan
└── Payment
```

The software structure matches the real business.

# Main Concepts of DDD

## 1. Domain

The **domain** is the business area your software solves.

Examples:

| Software | Domain |
|---|---|
| Amazon | E-commerce |
| Uber | Transportation |
| Hospital System | Healthcare |
| Banking App | Finance |

## 2. Domain Model

A **domain model** is a representation of real-world business concepts in software.

Example:

Banking domain:

```
Account
---------
accountNumber
balance

deposit()
withdraw()
transfer()
```

The model contains both:

- Data
- Business rules

## 3. Ubiquitous Language

This is one of the most important DDD ideas.

### Meaning:

> Developers and business experts should use the same language when discussing the system.

Example:

A bank employee says:

> "A customer transfers money from one account to another."

Developers should use the same terms:

```
Customer
Account
Transfer
Transaction
```

Not confusing technical names like:

```
UserObject
DataRecord
ProcessHandler
```

## 4. Entity

An **Entity** is an object that has a unique identity and changes over time.

Example:

A customer:

```
Customer ID: 101
Name: Rahul
```

Even if Rahul changes his name:

```
Customer ID: 101
Name: Rohan
```

It is still the same customer.

Examples:

- User
- Account
- Order
- Product

---

## 5. Value Object

A **Value Object** is an object defined only by its value, not identity.

Example:

Address:

```
Street: MG Road
City: Bangalore
```

```
PIN: 560001
```

Two addresses with the same values are considered equal.

Other examples:

- Money
- Date
- Coordinates
- Phone number

---

## 6. Aggregate

An **Aggregate** is a group of related objects treated as one unit.

Example:

Order system:

```
Order (Aggregate Root)
 │
 ├── Order Items
 │
 └── Payment Details
```

You do not directly modify order items.

You go through the Order:

```
Order.addItem()
Order.removeItem()
```

The aggregate protects business rules.

---

## 7. Aggregate Root

The main object that controls an aggregate.

Example:

Shopping cart:

```
ShoppingCart
      │
      ├── Product A
      ├── Product B
```

The cart controls adding/removing products.

---

## 8. Repository

A repository handles data storage operations.

Instead of directly writing database code:

```
SQL
SELECT * FROM customers
```

DDD uses:

```
Java
customerRepository.findById(101);
```

The domain does not care whether data comes from:

- MySQL
- MongoDB
- API
- File storage

---

## 9. Domain Service

Sometimes business logic does not naturally belong to one entity.

Example:

Money transfer:

```
Account A
    |
Transfer Service
    |
Account B
```

A transfer involves multiple accounts, so a service handles it.

---

## DDD Architecture

A common DDD structure:

```
Application Layer
        |
        |
Domain Layer
        |
        |
Infrastructure Layer
        |
        |
Database
```

## 1. Domain Layer

Contains:

- Business rules
- Entities
- Value objects
- Domain services

Example:

```
Account
Transaction
PaymentRule
```

## 2. Application Layer

Controls application flow.

Example:

```
TransferMoneyUseCase
```

It coordinates actions but does not contain business rules.

---

## 3. Infrastructure Layer

Handles technical details:

- Database
- APIs
- External services

Example:

```
MySQLCustomerRepository
PaymentGateway
EmailService
```

## DDD Example: Online Shopping

## Without DDD

Code may look like:

```
UserController
ProductController
OrderController

Database Tables
```

Business logic becomes scattered.

## With DDD

```
E-commerce Domain

Customer
 |
Order
 |
Product
 |
Payment
 |
Shipping
```

Each object understands its own rules.

Example:

Instead of:

```
Java
if(order.status=="paid"){
   ship(order);
}
```

DDD prefers:

```
Java
order.ship();
```

The Order object knows when shipping is allowed.

## DDD vs Traditional Layered Architecture

| Traditional Approach | DDD |
| --- | --- |
| Organized around technical layers | Organized around business concepts |
| Database often drives design | Business rules drive design |
| Logic may spread everywhere | Logic stays near domain objects |
| Good for simple applications | Good for complex systems |

## DDD and Microservices

DDD is often used with **microservices**.

Example:

An e-commerce application:

```
Order Service
    |
    Order Domain


Payment Service
    |
    Payment Domain


Inventory Service
    |
    Inventory Domain
```

Each microservice represents a business capability.

---

## Benefits of DDD

✅ Better understanding of business requirements
✅ Cleaner software structure
✅ Easier maintenance
✅ Better communication between developers and business teams
✅ Handles complex applications effectively
✅ Supports microservices architecture

---

## When should you use DDD?

DDD is useful for:

✅ Large systems
✅ Complex business rules
✅ Long-term projects
✅ Multiple development teams

Examples:

- Banking platforms
- Healthcare systems
- Insurance systems
- Enterprise applications

---

## When not to use DDD?

For simple applications:

- Calculator app
- Small website
- Simple CRUD application

DDD may add unnecessary complexity.

## DDD vs CRUD

| CRUD | DDD |
| --- | --- |
| Focuses on data operations | Focuses on business rules |
| Create, Read, Update, Delete | Model real-world concepts |
| Database-driven | Domain-driven |
| Good for simple apps | Good for complex systems |

Example:

CRUD:

```
Update Account Balance
```

DDD:

```
Account.withdraw(amount)
```

The account object decides whether withdrawal is allowed.

## Interview Definition

> **Domain-Driven Design (DDD) is a software development approach that focuses on modeling software around the business domain, using domain concepts, entities, value objects, and business rules to create maintainable and scalable systems.**

## Easy way to remember:

**Traditional design:**

> "How do I store and process data?"

**DDD design:**

> "What is the business doing, and how can software represent it?"

**In one sentence:**
**DDD = Build software that closely matches the real-world business it is designed to solve.**

> CQRS (Command Query Responsibility Segregation)

## CQRS (Command Query Responsibility Segregation)

**CQRS** is a software architecture pattern that separates **operations that change data** from **operations that read data**.

In simple words:

> **CQRS means using different models or paths for writing data and reading data.**

The main idea is:

- **Commands** → Change the system state (write operations)
- **Queries** → Retrieve information without changing anything (read operations)

---

## Why do we need CQRS?

In traditional applications, the same database model is often used for both:

- Updating data
- Reading data

Example:

```
Application
     |
     |
Single Model
     |
  Database
```

For small applications, this works well.

But in large systems, reading and writing have different needs:

- Writes need strong validation and business rules.
- Reads need fast searching and optimized data views.

CQRS separates them.

---

## Traditional CRUD Approach vs CQRS

## Traditional CRUD

One model handles everything:

```
User Request
     |
     |
  User Model
     |
  Database
```

Example:

```
User
- Create user
- Update user
- View user
- Search user
```

## CQRS Approach

Separate models:

```
             Application
                 |
        ------------------
        |                |
   Command Model     Query Model
        |                |
 Write Database    Read Database
```

## Command (Write Side)

A **command** is a request to change data.

Examples:

- Create an order
- Update customer details
- Transfer money
- Cancel a booking

A command says:

> "Do this action."

It does not return large amounts of data.

---

### Example

Command:

```JSON
{
  "command": "PlaceOrder",
  "productId": 101,
  "quantity": 2
}
```

The system:

1. Validates the request.
2. Applies business rules.
3. Updates the database.

# Query (Read Side)

A **query** retrieves data but does not modify anything.

Examples:

- Get user profile
- Search products
- View order history
- Check account balance

A query says:

> "Give me information."

---

### Example

Query:

```
GetOrderDetails(orderId=1001)
```

Response:

```JSON
{
 "orderId":1001,
 "status":"Delivered",
 "items":3
}
```

# Real-world Example: Banking System

A bank has two very different operations:

# Money Transfer (Command)

```
Transfer ₹5000 from Account A to Account B
```

Requires:

- Validation
- Security checks
- Transaction handling
- Business rules

---

# Account Balance Check (Query)

```
Show current balance
```

Requires:

- Fast data retrieval

---

With CQRS:

```
                Bank System

        ----------------------
        |                     |
  Transfer Service      Balance Service
    (Command)              (Query)

        |                     |
  Transaction DB       Read Database
```

# Example: E-commerce System

# Command side

Handles:

- Place order
- Cancel order
- Update inventory
- Process payment

Example:

```
PlaceOrderCommand
        |
Order Service
        |
Order Database
```

# Query side

Handles:

- Product search
- Order history
- Customer dashboard

Example:

```
SearchProductsQuery
        |
Search Service
```

```
        |
Read Database
```

## CQRS with Event Sourcing

CQRS is often combined with **Event Sourcing**.

Instead of storing only the latest state:

Traditional:

```
Account Balance = ₹5000
```

Event sourcing stores events:

```
Account Created
+
Deposited ₹10000
-
Withdrawn ₹5000
=
Current Balance ₹5000
```

The events become the source of truth.

---

## CQRS Flow

Example: Placing an order

```
User
 |
 |
Place Order Request
 |
 ▼
Command Handler
 |
 ▼
Business Logic
 |
 ▼
Write Database
 |
 ▼
Event Created
 |
 ▼
Read Database Updated
 |
 ▼
User Views Order
```

## Components of CQRS

## 1. Command Handler

Receives commands.

Example:

```
CreateOrderCommand
```

## 2. Domain Logic

Applies business rules.

Example:

```
Can this user place this order?
Is stock available?
```

## 3. Write Database

Stores transactional data.

Optimized for:

- Accuracy
- Consistency

---

## 4. Read Database

Stores data optimized for queries.

Optimized for:

- Speed
- Search
- Reports

---

# Advantages of CQRS

## 1. Better Performance

Read and write workloads can be optimized separately.

Example:

A shopping website has:

- Millions of product searches
- Fewer product updates

The read database can be scaled independently.

## 2. Scalability

You can add more servers for reading:

```
        Read Requests

Server 1
Server 2
Server 3
```

while keeping write operations separate.

## 3. Cleaner Business Logic

Commands contain rules for changing data.

Queries only retrieve data.

Responsibilities are clearer.

## 4. Better Security

You can control permissions separately.

Example:

- Customer can query order status.
- Only admin can modify orders.

# Disadvantages of CQRS

## 1. More Complexity

Instead of one model:

```
Single Model
```

you now manage:

```
Command Model
Query Model
Synchronization
```

## 2. Data Synchronization Issues

The read database may not update immediately.

Example:

User places an order:

```
Write DB:
Order created ✓

Read DB:
Updating...
```

For a short time, the user may not see the order.

This is called **eventual consistency**.

---

## 3. More Development Effort

Requires:

- More code
- More infrastructure
- Event handling

---

## CQRS vs CRUD

| CRUD | CQRS |
|---|---|
| Same model for read and write | Separate models |
| Simple architecture | More complex architecture |
| Good for small applications | Good for complex systems |
| Direct database operations | Command and query separation |
| Usually one database | Often separate read/write databases |

---

## CQRS vs REST API

They solve different problems:

**REST:**

- How applications communicate.

Example:

```
GET /users
POST /orders
```

**CQRS:**

- How application operations are organized internally.

Example:

```
CreateOrderCommand
GetOrderQuery
```

They can be used together.

---

## Where is CQRS used?

Common in:

- Banking systems
- Trading platforms
- E-commerce platforms
- Healthcare systems
- Enterprise applications
- Large-scale distributed systems

---

## When should you use CQRS?

Use CQRS when:

✅ Read and write workloads are very different
✅ Business rules are complex
✅ Application needs high scalability
✅ Multiple teams work on different parts

Avoid CQRS for:

❌ Simple CRUD applications
❌ Small projects
❌ Basic websites with simple data operations

---

## Interview Definition

> **CQRS (Command Query Responsibility Segregation) is an architectural pattern that separates data modification operations (commands) from data retrieval operations (queries), allowing each side to be optimized independently.**

---

## Easy way to remember:

**Command = Change something ✏️**
**Query = Ask something 🔍**

**CQRS = Separate writing data from reading data to build scalable and maintainable systems.**

> Event-Driven Architecture

## Event-Driven Architecture (EDA)

**Event-Driven Architecture (EDA)** is a software architecture pattern where applications communicate and react to **events** instead of directly calling each other.

In simple words:

> **An event-driven system works by producing, detecting, and responding to events that represent something that has happened.**

---

## What is an Event?

An **event** is a record of something that happened in the system.

An event is usually written in the past tense because it represents a completed action.

Examples:

- `OrderPlaced`
- `PaymentCompleted`
- `UserRegistered`
- `ProductAdded`
- `PackageDelivered`

Example event:

```JSON
{
  "event": "OrderPlaced",
  "orderId": 1001,
  "customerId": 50,
  "amount": 2500
}
```

This means:

> "An order has been placed."

---

## Traditional Architecture vs Event-Driven Architecture

## Traditional Request-Response Model

In a traditional system, services call each other directly.

Example: Online shopping:

```
Customer
   |
   ▼
Order Service
   |
   ├── Payment Service
   |
   ├── Inventory Service
   |
   └── Email Service
```

The Order Service must know about all other services.

Problems:

- Tight coupling
- Harder to modify
- One service failure may affect others

---

# Event-Driven Architecture

Services communicate through events:

```
                Event Broker
                     |
        -----------------------------
        |            |              |
        ▼            ▼              ▼
Payment         Inventory       Notification
Service         Service          Service

             ▲
             |
         Order Service
```

The Order Service only publishes:

```
OrderPlaced
```

Other services decide whether they need to react.

---

# Main Components of Event-Driven Architecture

## 1. Event Producer (Publisher)

A component that creates events.

Examples:

- Order Service
- Payment Service

- User Service

Example:

```
Customer places order

Order Service
      |
      ▼
OrderPlaced Event
```

## 2. Event Broker (Message Broker)

The middle layer that receives and distributes events.

It acts like a communication channel.

Examples:

- Apache Kafka
- RabbitMQ
- Amazon Simple Notification Service

The broker:

- Stores events.
- Delivers events.
- Manages communication between services.

---

## 3. Event Consumer (Subscriber)

A service that listens for specific events and reacts.

Example:

When it receives:

```
OrderPlaced
```

Different consumers respond:

```
Payment Service
→ Process payment

Inventory Service
→ Reduce stock

Email Service
→ Send confirmation email
```

## Real-World Example: Amazon Order System

Imagine you buy a laptop.

# Step 1: Order Created

Customer clicks:

```
Buy Now
```

Order Service creates:

```
OrderPlaced Event
```

# Step 2: Multiple services react

## Payment Service

Receives:

```
OrderPlaced
```

Action:

```
Charge customer
```

Creates:

```
PaymentCompleted Event
```

## Inventory Service

Receives:

```
OrderPlaced
```

Action:

```
Decrease laptop stock
```

## Notification Service

Receives:

```
PaymentCompleted
```

Action:

```
Send email/SMS confirmation
```

# Types of Event-Driven Architecture

## 1. Event Notification

The event only says that something happened.

Example:

```
UserRegistered
```

The consumer fetches details if needed.

Flow:

```
Event
 ↓
Consumer
 ↓
Get additional data
```

## 2. Event-Carried State Transfer

The event contains the required data.

Example:

```
JSON
{
 "event": "UserUpdated",
 "id":101,
 "name":"Rahul",
 "email":"rahul@gmail.com"
}
```

The consumer does not need another request.

---

## 3. Event Sourcing

Instead of storing only the current state, the system stores all events.

Example:

Bank account:

Instead of:

```
Balance = ₹5000
```

Store:

```
AccountCreated
Deposited ₹10000
Withdrawn ₹5000
```

Current balance is calculated from events.

---

## Event-Driven Architecture with Microservices

EDA is commonly used with microservices.

Example:

```
                Kafka
                  |
    -------------------------------
    |              |               |
Order Service  Payment Service  Shipping Service
```

Each service:

- Has its own responsibility.
- Communicates through events.
- Is loosely coupled.

---

## Benefits of Event-Driven Architecture

## 1. Loose Coupling

Services do not directly depend on each other.

Example:

Order Service does not need to know:

- Who sends emails.
- Who updates inventory.

It only publishes:

```
OrderPlaced
```

## 2. Scalability

You can add more consumers when traffic increases.

Example:

During a sale:

```
More orders
     |
     ▼
More Payment Service instances
```

## 3. Better Fault Isolation

If email service fails:

```
Order Service ✅
Payment Service ✅
Inventory Service ✅
Email Service ❌
```

Orders can continue.

---

## 4. Real-Time Processing

Events can be processed immediately.

Examples:

- Fraud detection
- Stock updates
- Notifications
- Recommendations

---

# Challenges of Event-Driven Architecture

## 1. Complexity

Managing:

- Events
- Message queues
- Failures
- Event ordering

can become difficult.

---

## 2. Eventual Consistency

Data may not update everywhere immediately.

Example:

Order placed:

```
Order Database:
Created ✓
```

```
Inventory Database:
Updating...
```

For a short time, systems may show different states.

## 3. Debugging Difficulty

A single user action may create many events:

```
OrderPlaced
    ↓
PaymentCompleted
    ↓
InventoryUpdated
    ↓
ShipmentCreated
```

Tracing problems requires good monitoring.

## Event-Driven Architecture vs REST API

| REST API | Event-Driven Architecture |
|---|---|
| Request-response communication | Event-based communication |
| Client asks for action | System reacts to events |
| Usually synchronous | Often asynchronous |
| Direct service communication | Loosely coupled communication |
| Example: GET /products | Example: ProductCreated event |

They are often used together.

Example:

A mobile app uses REST APIs, while backend services communicate using events.

## Event-Driven Architecture vs CQRS

They are related but different.

| CQRS | Event-Driven Architecture |
|---|---|
| Separates commands and queries | Uses events for communication |
| Focuses on data operations | Focuses on system communication |
| Often uses events | May or may not use CQRS |

A system can use both:

```
Command
   |
   ▼
```

```
Write Model
   |
   ▼
Event
   |
   ▼
Read Model
```

# Real-World Applications

## Banking

Events:

```
MoneyTransferred
PaymentCompleted
FraudDetected
```

Used for:

- Transaction processing
- Fraud detection
- Notifications

---

## E-commerce

Events:

```
OrderPlaced
ProductPurchased
ShipmentCreated
```

Used for:

- Inventory
- Payment
- Shipping

---

## Ride-sharing apps

Events:

```
RideRequested
DriverAssigned
RideCompleted
```

Used for:

- Matching drivers
- Payments
- Notifications

---

## Social media

Events:

```
PostCreated
CommentAdded
LikeAdded
```

Used for:

- Feeds
- Notifications
- Recommendations

---

## Interview Definition

> **Event-Driven Architecture is a software architecture pattern where components communicate through events. Producers generate events, and consumers react to those events asynchronously, enabling scalable and loosely coupled systems.**

---

## Easy way to remember:

**Traditional system:**

> "Service A calls Service B."

**Event-driven system:**

> "Something happened; whoever cares can react."

**In one sentence:**

**Event-Driven Architecture = Build systems where actions produce events, and different services independently respond to those events.**

> Serverless Architecture

## Serverless Architecture

**Serverless Architecture** is a software architecture where developers build and run applications **without managing physical servers or traditional server infrastructure**.

In simple words:

> **Serverless means you write and deploy code, while a cloud provider manages the servers, scaling, maintenance, and infrastructure.**

Despite the name **"serverless"**, servers still exist. The difference is that **you don't have to manage them**.

---

## Traditional Server-Based Architecture

In a traditional application:

```
User
 |
 ▼
Web Server
 |
 ▼
Application Server
 |
 ▼
Database
```

The company is responsible for:

- Buying servers
- Installing software
- Updating servers
- Managing security
- Scaling during high traffic
- Maintaining uptime

Example:

A company runs its own server:

```
Physical Machine
      |
Operating System
      |
Web Application
      |
Database
```

## Serverless Architecture

In serverless:

```
User
 |
 ▼
```

```
Cloud Provider
 |
 ▼
Function Executes
 |
 ▼
Database / Services
```

The developer only focuses on:

- Writing code
- Deploying functions
- Building features

The cloud provider handles:

- Servers
- Operating systems
- Scaling
- Availability
- Infrastructure maintenance

---

## How Serverless Works

The basic idea:

1. An event happens.
2. A function is triggered.
3. The function runs.
4. The result is returned.
5. Resources are released.

Example:

User uploads an image:

```
User uploads photo
        |
        ▼
Image Upload Event
        |
        ▼
Serverless Function
        |
        ▼
Resize Image
        |
        ▼
Store Image
```

The function only runs when needed.

---

# Main Components of Serverless Architecture

## 1. Function as a Service (FaaS)

The core idea of serverless.

A developer writes small functions that run when triggered.

Examples:

- AWS Lambda
- Azure Functions
- Google Cloud Functions

Example:

```JavaScript
function calculateTax(price) {
    return price * 0.18;
}
```

The cloud runs this function when needed.

---

## 2. Event Trigger

A serverless function usually starts because of an event.

Examples:

| Event | Action |
| --- | --- |
| User clicks button | Run function |
| File uploaded | Process file |
| Payment completed | Send notification |
| Timer reached | Run scheduled task |
| Database updated | Trigger processing |

---

## 3. Managed Services

Serverless applications often use cloud-managed services:

Examples:

- Databases
- Storage
- Authentication

- Messaging
- APIs

The provider manages the infrastructure.

---

## Real-World Example: Online Shopping Website

Imagine an e-commerce application.

## Traditional approach:

```
User
 |
Web Server
 |
Application Server
 |
Database
 |
Storage Server
```

The company manages everything.

---

## Serverless approach:

```
Customer places order
          |
          ▼
API Gateway
          |
          ▼
Order Function
          |
          ▼
Database
          |
          ▼
Notification Function
          |
          ▼
Email Sent
```

Different functions handle different tasks:

- Create order
- Process payment
- Update inventory
- Send email

---

## Example: Image Processing App

A user uploads an image.

Flow:

```
Upload Image
      |
      ▼
Cloud Storage
      |
      ▼
Trigger Function
      |
      ▼
Resize Image
      |
      ▼
Save New Image
```

The image-processing code runs only when an image is uploaded.

---

## Advantages of Serverless Architecture

## 1. No Server Management

Developers do not manage:

- Hardware
- Operating systems
- Server updates

They focus on application logic.

---

## 2. Automatic Scaling

Serverless automatically adjusts resources.

Example:

Normal day:

```
100 users
↓
2 function instances
```

During a sale:

```
1 million users
↓
Thousands of function executions
```

## 3. Cost Efficiency

You usually pay only for actual usage.

Traditional:

```
Server running 24/7
= Pay even when unused
```

Serverless:

```
Function runs for 2 seconds
= Pay for 2 seconds
```

## 4. Faster Development

Developers can quickly build and deploy features without infrastructure setup.

---

## 5. High Availability

Cloud providers handle:

- Server failures
- Data center issues
- Infrastructure recovery

---

# Disadvantages of Serverless Architecture

## 1. Cold Start Problem

When a function has not run for a while, the cloud provider may need time to start it.

Example:

First request:

```
Request arrives
      |
      ▼
Create execution environment
      |
      ▼
Run function
```

This causes a small delay.

---

## 2. Vendor Lock-in

Applications may depend heavily on one cloud provider.

Example:

Moving from one provider's serverless platform to another may require changes.

---

## 3. Limited Execution Time

Many serverless functions have maximum execution limits.

Example:

A function may not be suitable for:

- Long video processing
- Large computations

---

## 4. Debugging Complexity

Distributed serverless systems can be harder to debug because many small functions interact.

---

## Serverless vs Traditional Servers

| Traditional Servers | Serverless |
|---|---|
| Developer manages servers | Cloud provider manages servers |
| Fixed server capacity | Automatic scaling |
| Pay for running servers | Pay for usage |
| Long-running applications | Short-lived functions |
| More infrastructure control | Less infrastructure control |

---

## Serverless vs Microservices

They are related but different.

| Microservices | Serverless |
|---|---|
| Splits application into services | Runs code as functions |
| Services may run continuously | Functions run when triggered |
| You may manage servers | Cloud manages infrastructure |
| Architecture style | Deployment/hosting model |

They can be combined:

```
Microservices
      |
```

```
      ▼
Serverless Functions
```

Example:

An order microservice may be implemented using serverless functions.

---

## Common Serverless Use Cases

### 1. APIs

Example:

Mobile app calls:

```
GET /products
```

A serverless function retrieves products.

---

### 2. File Processing

Examples:

- Resize images
- Convert videos
- Generate thumbnails

---

### 3. Notifications

Examples:

- Send emails
- Send SMS
- Push notifications

---

### 4. Data Processing

Examples:

- Analyze logs
- Process streams
- Generate reports

## 5. Scheduled Tasks

Examples:

- Daily backups
- Monthly reports
- Cleanup jobs

## Popular Serverless Platforms

| Platform | Provider |
|---|---|
| AWS Lambda | Amazon Web Services |
| Azure Functions | Microsoft Azure |
| Google Cloud Functions | Google Cloud |
| Cloudflare Workers | Cloudflare |
| Vercel Functions | Vercel |

## Serverless Architecture with Other Patterns

Serverless is often combined with:

### Microservices

```
User Service
     |
Serverless Function
```

### Event-Driven Architecture

```
Event
 |
 ▼
Serverless Function
 |
 ▼
Action
```

### API Gateway

```
Mobile App
     |
     ▼
API Gateway
     |
     ▼
Serverless Function
```

## Interview Definition

> **Serverless Architecture is a cloud computing model where developers build and run applications without managing servers. Cloud providers automatically handle infrastructure, scaling, and resource allocation while developers focus on writing application code.**

---

## Easy way to remember:

**Traditional:**

> "I manage servers and run my application."

**Serverless:**

> "I write functions; the cloud runs them when needed."

**In one sentence:**

**Serverless Architecture = Build applications using cloud-managed functions where infrastructure management is handled automatically by the provider.**

CI/CD (Continuous Integration / Continuous Deployment)

## CI/CD (Continuous Integration / Continuous Deployment)

**CI/CD** is a set of software development practices that automate the process of **building, testing, and delivering software changes** quickly and reliably.

CI/CD stands for:

- **CI → Continuous Integration**
- **CD → Continuous Delivery / Continuous Deployment**

In simple words:

> **CI/CD means automatically testing and delivering code changes whenever developers make updates, reducing manual work and errors.**

---

## Why do we need CI/CD?

In traditional software development:

```
Developer writes code
        |
        ▼
Manual testing
        |
        ▼
```

```
Manual deployment
        |
        ▼
Release to users
```

Problems:

- Takes a lot of time.
- Human errors can happen.
- Bugs may be discovered late.
- Releases become risky.

With CI/CD:

```
Developer writes code
        |
        ▼
Push code
        |
        ▼
Automatic Build
        |
        ▼
Automatic Tests
        |
        ▼
Automatic Deployment
        |
        ▼
Users get updates
```

# Part 1: Continuous Integration (CI)

## What is CI?

**Continuous Integration** is the practice of frequently merging developers' code changes into a shared repository and automatically testing them.

Simple meaning:

> **Developers regularly add their code to the main project, and automated systems check whether it works.**

---

## Example of CI

Imagine a team of developers:

```
Developer A → Login feature
Developer B → Payment feature
Developer C → Profile feature
```

Each developer writes code and pushes it:

```
Developer
    |
```

```
    ▼
Git Repository
    |
    ▼
CI Pipeline
    |
    ├── Build Application
    ├── Run Unit Tests
    ├── Check Code Quality
    └── Report Results
```

If tests pass:

```
✅ Code accepted
```

If tests fail:

```
❌ Developer fixes the issue
```

# CI Activities

# 1. Code Commit

Developer uploads code:

```
Bash
git push
```

# 2. Build

The system compiles the application.

Example:

```
Source Code
     |
     ▼
Compiler
     |
     ▼
Application Build
```

# 3. Automated Testing

The system runs:

- Unit tests
- Integration tests
- Security tests

Example:

```
Test Login
Test Payment
Test Database Connection
```

## 4. Code Quality Checks

Checks:

- Code style
- Security issues
- Bugs

---

# Part 2: Continuous Delivery (CD)

## What is Continuous Delivery?

Continuous Delivery means:

> **The software is always ready to be released after passing automated tests.**

The deployment process is automated, but a human may approve the final release.

Flow:

```
Code
 |
Build
 |
Test
 |
Ready for Production
 |
Manual Approval
 |
Release
```

# Part 3: Continuous Deployment (CD)

## What is Continuous Deployment?

Continuous Deployment goes one step further:

> **Every change that passes all tests is automatically released to production.**

No manual approval.

Flow:

```
Developer Push
      |
      ▼
```

```
Automated Tests
      |
      ▼
Production Release
      |
      ▼
Users Receive Update
```

## Continuous Delivery vs Continuous Deployment

| Continuous Delivery | Continuous Deployment |
|---|---|
| Automatically prepares releases | Automatically releases software |
| Requires manual approval | No manual approval |
| Safer for controlled environments | Faster releases |
| Common in enterprise systems | Common in cloud applications |

---

## Real-World Example: Netflix

Netflix deploys software frequently.

A developer changes recommendation logic:

```
Developer writes code
        |
        ▼
CI Pipeline
        |
        ├── Build
        ├── Test
        └── Security checks
        |
        ▼
Deployment
        |
        ▼
Users get improved recommendations
```

The process happens automatically.

---

## CI/CD Pipeline

A typical CI/CD pipeline looks like:

```
          CI
-------------------
Code Commit
     |
     ▼
  Build
     |
     ▼
  Test
     |
     ▼
Code Quality Check
```

```
          CD
------------------
     |
     ▼
Deploy to Staging
     |
     ▼
Deploy to Production
     |
     ▼
   Monitor
```

## Components of CI/CD

## 1. Source Control

Stores code and tracks changes.

Examples:

- Git
- GitHub
- GitLab
- Bitbucket

---

## 2. Build Tools

Convert source code into a runnable application.

Examples:

- Maven (Java)
- Gradle (Java)
- npm (JavaScript)
- CMake (C++)

---

## 3. Testing Tools

Automatically verify code.

Examples:

- JUnit
- pytest
- Selenium

---

## 4. CI/CD Tools

Automate pipelines.

Popular tools:

- Jenkins
- GitHub Actions
- GitLab CI/CD
- CircleCI

---

# Example: Web Application Deployment

Suppose you build an online shopping website.

A developer changes the checkout page.

### Step 1: Developer commits code

```
git commit
git push
```

### Step 2: CI starts

Automatically:

```
Install dependencies
        |
        ▼
Build application
        |
        ▼
Run tests
```

### Step 3: CD starts

If tests pass:

```
Deploy to server
        |
        ▼
New checkout page available
```

# Benefits of CI/CD

## 1. Faster Development

Developers can release features quickly.

---

## 2. Fewer Bugs

Automated tests catch problems early.

---

## 3. Smaller Releases

Instead of releasing huge updates:

```
Large update every 6 months
```

You get:

```
Small updates every day/week
```

## 4. Better Collaboration

Multiple developers can work safely.

---

## 5. Reliable Deployment

The same automated process is used every time.

---

# Challenges of CI/CD

## 1. Initial Setup Complexity

Creating pipelines requires time.

---

## 2. Test Quality Matters

Bad tests create false confidence.

---

## 3. Infrastructure Cost

Requires:

- Build servers
- Testing environments

- Monitoring systems

---

## CI/CD and DevOps

CI/CD is a major part of **DevOps**.

DevOps combines:

- Development (Developers)
- Operations (System administrators)

Goal:

> Build, test, and release software faster through automation and collaboration.

Relationship:

```
DevOps
  │
  ├── CI/CD
  ├── Automation
  ├── Monitoring
  └── Infrastructure Management
```

## CI/CD vs Agile

| Agile | CI/CD |
|---|---|
| Development methodology | Automation practice |
| Focuses on iterative development | Focuses on software delivery |
| Uses sprints | Uses automated pipelines |

---

## Interview Definition

> **CI/CD is a software engineering practice that automates the process of integrating code changes, testing applications, and deploying software to production. Continuous Integration focuses on frequent code integration and testing, while Continuous Deployment automates releasing changes to users.**

---

## Easy way to remember:

**CI:**

> "Does my new code work with existing code?"

**CD:**

"Can I safely deliver this code to users?"

**In one sentence:**

**CI/CD = Automatically build, test, and deliver software changes quickly and reliably.**

> TDD (Test-Driven Development)

# TDD (Test-Driven Development)

**TDD (Test-Driven Development)** is a software development approach where developers **write tests before writing the actual code**.

In simple words:

> **First write a test that describes what the code should do, then write the code to make that test pass.**

The main idea is:

**Test → Code → Improve**

---

# Traditional Development vs TDD

# Traditional Approach

Usually developers work like this:

```
Requirement
     |
     ▼
Write Code
     |
     ▼
Test Code
     |
     ▼
Fix Bugs
```

Testing happens after implementation.

---

# TDD Approach

TDD reverses the process:

```
Write Test
     |
     ▼
Write Code
     |
     ▼
Run Test
```

```
     |
     ▼
Improve Code
```

The test guides the design of the code.

---

## The TDD Cycle: Red → Green → Refactor

TDD follows a repeating three-step cycle.

```
       RED
        |
        ▼
  Write failing test
        |
        ▼
      GREEN
        |
        ▼
Write minimum code
to pass the test
        |
        ▼
   REFACTOR
        |
        ▼
Improve code quality
```

## 1. Red Phase (Write a Failing Test)

First, create a test for a feature that does not exist yet.

Example:

Requirement:

> Create a function that adds two numbers.

Write test:

```Python
def test_add():
    result = add(2, 3)
    assert result == 5
```

At this point:

```
Test fails ❌
```

because the `add()` function does not exist.

This is the **Red phase**.

---

## 2. Green Phase (Make the Test Pass)

Now write the simplest code possible.

```
Python
def add(a, b):
    return a + b
```

Run the test:

```
Test passes ✅
```

This is the **Green phase**.

---

## 3. Refactor Phase (Improve Code)

Now improve the code without changing behavior.

Example:

Before:

```
Python
def add(a,b):
    return a+b
```

After:

```
Python
def add(first_number, second_number):
    return first_number + second_number
```

Run tests again:

```
Tests still pass ✅
```

This is the **Refactor phase**.

---

## Real-World Example: Banking System

Requirement:

> A customer should not withdraw more money than their balance.

---

## Step 1: Write Test First

```
Python
def test_withdraw_more_than_balance():
    account = Account(1000)

    result = account.withdraw(1500)

    assert result == "Insufficient Balance"
```

The test fails because withdrawal logic is not implemented.

## Step 2: Write Code

```
Python
class Account:

    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            return "Insufficient Balance"

        self.balance -= amount
```

Test passes.

## Step 3: Refactor

Improve the design:

```
Python
class Account:

    def withdraw(self, amount):
        self.validate_withdrawal(amount)
        self.balance -= amount
```

The behavior remains the same.

## Types of Tests Used in TDD

## 1. Unit Tests

Test a small piece of code.

Example:

Testing a calculator function:

```
calculateTax()
```

## 2. Integration Tests

Test multiple components working together.

Example:

```
Application
     |
Database
```

## 3. Acceptance Tests

Test whether the application satisfies business requirements.

Example:

```
Customer can successfully complete checkout
```

## TDD Example in Web Application

Feature:

"User registration"

## Test first:

```
Given:
A new user

When:
User registers with email

Then:
Account should be created
```

## Implementation:

```
Registration Service
        |
        ▼
User Database
```

## Refactoring:

Improve:

- Validation
- Error handling
- Code structure

---

## Benefits of TDD

## 1. Fewer Bugs

Since every feature has tests, problems are found early.

---

## 2. Better Code Design

Writing tests first forces developers to think about:

- What the code should do.
- How components interact.
- How code will be used.

---

## 3. Safer Refactoring

Developers can improve code without fear.

Example:

```
Change code
     |
Run tests
     |
Everything works ✅
```

## 4. Living Documentation

Tests explain how the system should behave.

Example:

```
Python
test_user_can_login()
test_user_cannot_login_with_wrong_password()
```

The test names describe requirements.

---

## 5. Faster Debugging

When a test fails, developers know which behavior broke.

---

## Disadvantages of TDD

## 1. More Initial Development Time

Writing tests first takes extra effort.

## 2. Requires Practice

Poorly written tests can become difficult to maintain.

## 3. Not Every Situation Fits

Some areas are difficult to test first:

- User interface design
- Exploratory programming
- Complex hardware interactions

## TDD vs Traditional Testing

| TDD | Traditional Testing |
|---|---|
| Test written before code | Test written after code |
| Test drives design | Code drives testing |
| Continuous testing | Testing phase later |
| Encourages small changes | Often larger changes |

## TDD vs BDD

| TDD | BDD |
|---|---|
| Developer-focused | Business-focused |
| Tests functions/classes | Tests user behavior |
| Uses technical language | Uses business language |

Example:

TDD:

```
Python
test_calculate_discount()
```

BDD:

```
Given a customer has a premium account
When they buy a product
Then they receive a discount
```

# TDD and CI/CD

TDD works well with CI/CD.

Flow:

```
Developer writes test
        |
        ▼
Write code
        |
        ▼
Commit code
        |
        ▼
CI runs tests automatically
        |
        ▼
Deploy if successful
```

# TDD and SOLID Principles

TDD encourages good design practices:

### Single Responsibility Principle

Small classes are easier to test.

### Dependency Injection

Dependencies can be replaced with test objects.

### Loose Coupling

Independent components are easier to test.

---

# Where TDD is Used

Common in:

- Banking software
- Enterprise applications
- Backend systems
- APIs
- Financial applications
- Critical business logic

---

# Interview Definition

> **Test-Driven Development (TDD) is a software development methodology where developers write automated tests before implementing functionality. The development process follows a Red-Green-Refactor cycle to produce reliable and maintainable code.**

---

## Easy way to remember:

**Normal development:**

> Code first → Test later

**TDD:**

> Test first → Code → Improve

**In one sentence:**

**TDD = Let tests guide the design and development of your software.**

> OSI Model (Open Systems Interconnection)

## OSI Model (Open Systems Interconnection)

The **OSI Model** is a conceptual framework that explains **how data moves from one computer to another over a network**.

It divides network communication into **7 different layers**, where each layer has a specific responsibility.

In simple words:

> **OSI Model is a blueprint that helps understand how devices communicate over a network by breaking communication into seven layers.**

---

## Why was the OSI Model created?

Before OSI:

- Different companies created different networking systems.
- Devices from different manufacturers had compatibility problems.
- Troubleshooting networks was difficult.

The OSI model was created to provide a **standard way** to understand and design network communication.

It was developed by the **International Organization for Standardization (ISO)** in 1984.

---

## OSI Model Layers

The OSI model has **7 layers**:

```
 id="r7kq3d"
+---------------+
| 7. Application |
+---------------+
| 6. Presentation|
+---------------+
| 5. Session     |
+---------------+
| 4. Transport   |
+---------------+
| 3. Network     |
+---------------+
| 2. Data Link   |
+---------------+
| 1. Physical    |
+---------------+
```

A common way to remember:

**A**ll
**P**eople
**S**eem
**T**o
**N**eed
**D**ata
**P**rocessing

(Application → Presentation → Session → Transport → Network → Data Link → Physical)

---

## Layer 1: Physical Layer

## Purpose

The Physical layer is responsible for transmitting **raw bits (0s and 1s)** through physical media.

It deals with:

- Electrical signals
- Radio signals
- Cables
- Connectors
- Hardware

---

## Data Unit

**Bits**

Example:

```
101010101010
```

## Devices

- Cables
- Hubs
- Repeaters
- Network Interface Cards (NIC)

---

## Examples

- Ethernet cable
- Fiber optic cable
- Wi-Fi radio signals

---

## Real-world example

When you send a message:

```
Computer
   |
Electrical signals
   |
Network cable
```

The Physical layer sends the actual signals.

---

## Layer 2: Data Link Layer

## Purpose

The Data Link layer provides **node-to-node communication**.

It:

- Detects errors.
- Controls data flow.
- Creates frames.
- Uses MAC addresses.

---

## Data Unit

**Frame**

---

## Devices

- Switches
- Bridges

---

## Address Used

**MAC Address**

Example:

```
00:1A:2B:3C:4D:5E
```

## Protocols

- Ethernet
- Wi-Fi (802.11)

---

## Real-world example

A switch decides:

> "Which device on this local network should receive this data?"

---

## Layer 3: Network Layer

## Purpose

The Network layer handles:

- Logical addressing
- Routing
- Finding the best path between networks

---

## Data Unit

**Packet**

---

## Address Used

**IP Address**

Example:

```
192.168.1.10
```

## Devices

- Routers

---

## Protocols

- IP (Internet Protocol)
- ICMP

---

## Real-world example

When you open a website:

```
Your Computer
      |
      ▼
Router
      |
      ▼
Internet
      |
      ▼
Website Server
```

The Network layer finds the route.

---

## Layer 4: Transport Layer

## Purpose

The Transport layer provides **end-to-end communication**.

It handles:

- Reliability
- Error recovery
- Data segmentation
- Flow control

---

## Data Unit

**Segment**

---

## Main Protocols

## TCP (Transmission Control Protocol)

Provides:

- Reliable delivery
- Error checking
- Ordered packets

Used for:

- Websites
- Emails
- File transfers

---

## UDP (User Datagram Protocol)

Provides:

- Faster communication
- Less overhead

Used for:

- Online gaming
- Video calls
- Streaming

---

## Example

Sending a large file:

```
Original File
     ↓
Split into small pieces
     ↓
Transport Layer sends segments
```

# Layer 5: Session Layer

## Purpose

The Session layer manages communication sessions between applications.

It handles:

- Creating sessions
- Maintaining sessions
- Ending sessions

---

## Example

Video call:

```
Start Call
    |
Maintain Connection
    |
End Call
```

## Functions

- Authentication
- Session recovery
- Synchronization

---

# Layer 6: Presentation Layer

## Purpose

The Presentation layer converts data into a format that applications can understand.

It handles:

- Data encryption
- Data compression
- Data translation

---

# Examples

## Encryption

HTTPS:

```
Normal Data
     ↓
Encrypted Data
```

## Compression

Large file:

```
100 MB file
      ↓
20 MB compressed file
```

# Formats

Examples:

- JPEG
- PNG
- JSON
- XML
- ASCII

---

# Layer 7: Application Layer

# Purpose

The Application layer provides network services directly to user applications.

It is the layer closest to the user.

---

# Protocols

## HTTP/HTTPS

Used for websites.

Example:

```
Browser → Website Server
```

## FTP

File transfer.

---

## SMTP

Email sending.

---

## DNS

Converts domain names into IP addresses.

Example:

```
google.com
      ↓
142.250.x.x
```

# Complete Data Flow Example

Imagine you open a website:

```
https://example.com
```

# Sending request:

```
Application Layer
↓
Creates HTTP request

Presentation Layer
↓
Encrypts data (HTTPS)

Session Layer
↓
Creates connection

Transport Layer
↓
Uses TCP

Network Layer
↓
```

```
Adds IP addresses

Data Link Layer
↓
Creates frames

Physical Layer
↓
Sends electrical/radio signals
```

## OSI Encapsulation

When data moves down the OSI layers, each layer adds its own information.

Example:

```
Application
    Data

Transport
    Segment

Network
    Packet

Data Link
    Frame

Physical
    Bits
```

This process is called **encapsulation**.

---

## Decapsulation

When data reaches the receiver:

```
Bits
 ↓
Frame
 ↓
Packet
 ↓
Segment
 ↓
Data
```

Each layer removes its information.

This is called **decapsulation**.

---

## OSI Model vs TCP/IP Model

The Internet mainly uses the TCP/IP model.

| OSI Model | TCP/IP Model |
|---|---|
| 7 Layers | 4 Layers |
| Theoretical reference model | Practical Internet model |

| OSI Model | TCP/IP Model |
| --- | --- |
| Developed by ISO | Developed for ARPANET/Internet |

Mapping:

```
OSI                 TCP/IP

Application
Presentation  ┐
Session       ├──  Application
              ┘

Transport     ──   Transport

Network       ──   Internet

Data Link
Physical      ┐
              ┘──  Network Access
```

# OSI Model and Troubleshooting

Network engineers use OSI layers to find problems.

Example:

# Problem:

"No internet connection"

Check layers:

### Layer 1:

Is the cable connected?

### Layer 2:

Is the network adapter working?

### Layer 3:

Does the device have an IP address?

### Layer 4:

Are ports blocked?

### Layer 7:

Is the application working?

---

# Devices by OSI Layer

| Layer | Device |
|---|---|
| Layer 7 | Proxy Server |
| Layer 6 | Encryption Software |
| Layer 5 | Session Services |
| Layer 4 | Load Balancer |
| Layer 3 | Router |
| Layer 2 | Switch |
| Layer 1 | Hub, Cable |

---

# OSI Model in Real Applications

## Web Browsing

Uses:

- Layer 7 → HTTP
- Layer 6 → TLS encryption
- Layer 4 → TCP
- Layer 3 → IP
- Layer 2 → Ethernet/Wi-Fi
- Layer 1 → Signals

---

## Video Streaming

Uses:

- Application → Streaming protocols
- Transport → TCP/UDP
- Network → IP routing
- Physical → Internet connection

---

# Interview Definition

> **The OSI (Open Systems Interconnection) model is a seven-layer conceptual framework that standardizes network communication by dividing the process of sending and receiving data into seven independent layers, each responsible for specific networking functions.**

---

## Easy Way to Remember the Layers

From top to bottom:

**Application** → What the user uses
**Presentation** → Format and security
**Session** → Connection management
**Transport** → Reliable delivery
**Network** → Routing and IP
**Data Link** → Local delivery and MAC
**Physical** → Signals and hardware

**In one sentence:**

**OSI Model = A 7-layer roadmap that explains how data travels from one device to another across a network.**

> SOA (Service-Oriented Architecture)

# SOA (Service-Oriented Architecture)

**SOA (Service-Oriented Architecture)** is a software architecture style where an application is built as a collection of **independent, reusable services** that communicate with each other through standard protocols.

In simple words:

> **SOA means breaking a large application into separate business services that can work independently and be reused by different applications.**

---

## Why do we need SOA?

In older systems, applications were often built as a single large block:

```
 id="v8m1zp"
+---------------------------+
|     Large Application     |
|                           |
| User Management           |
| Payment                   |
| Inventory                 |
| Reports                   |
| Notifications             |
+---------------------------+
```

Problems:

- Difficult to modify
- Hard to reuse features
- Changes in one area affect the whole system
- Difficult integration with other systems

SOA solves this by separating functionality into services.

## SOA Architecture

A typical SOA system looks like:


Each service performs a specific business function.

## What is a Service in SOA?

A **service** is a self-contained software component that provides a specific business capability.

Examples:

### Customer Service

Handles:

- Create customer
- Update customer
- Get customer details

### Payment Service

Handles:

- Payment processing
- Refunds
- Transaction history

### Inventory Service

Handles:

- Product availability
- Stock updates

# Key Principles of SOA

## 1. Loose Coupling

Services should have minimum dependency on each other.

Example:

Payment Service does not need to know how Order Service is implemented.

It only needs:

```
"Process Payment"
```

## 2. Service Reusability

A service can be used by multiple applications.

Example:

A payment service can be used by:

```
 id="b4o0o4"
Website
   |
Mobile App
   |
Partner Application
        |
        ▼
 Payment Service
```

## 3. Service Autonomy

Each service controls its own logic and resources.

Example:

Payment Service manages:

- Payment rules
- Transaction processing
- Security checks

---

## 4. Standardized Communication

Services communicate using common standards.

Examples:

- HTTP

- SOAP
- REST
- XML
- JSON

---

## 5. Discoverability

Services should be easy to find and understand.

A service should provide information like:

```
Service Name:
Payment Service

Purpose:
Process customer payments

Operations:
- Make Payment
- Refund Payment
```

## SOA Components

## 1. Service Provider

The system that creates and exposes a service.

Example:

A bank provides:

```
Payment Service
```

## 2. Service Consumer

The application that uses the service.

Example:

An online shopping website uses:

```
Payment Service
```

## 3. Service Registry

A directory where services are registered and discovered.

Flow:

```
Provider
   |
   ▼
Service Registry
   |
   ▼
Consumer Finds Service
```

Example:

A company has hundreds of services; the registry helps applications find them.

---

## SOA Communication Flow

Example: Online Shopping

Customer buys a product.

```
Customer
   |
   ▼
Order Application
   |
   ▼
Order Service
   |
   ▼
Payment Service
   |
   ▼
Inventory Service
   |
   ▼
Shipping Service
```

Each service performs its own task.

---

## Real-World Example: Banking System

A bank may have:

## Customer Service

```
Create Account
Update Customer Information
```

## Account Service

```
Check Balance
Manage Accounts
```

## Payment Service

```
Transfer Money
Process Payments
```

## Notification Service

```
Send SMS
Send Email
```

Different applications can reuse these services.

Example:

```
Mobile Banking App
        |
        ▼
      SOA Services
        |
        ▼
 Core Banking System
```

## SOA Technologies

## SOAP (Simple Object Access Protocol)

A common SOA communication protocol.

Features:

- XML-based
- Strict standards
- Enterprise-focused

Example SOAP message:

```xml
<Payment>
   <Amount>5000</Amount>
</Payment>
```

## REST APIs

Modern SOA systems often use REST.

Example:

```
POST /payment
```

Request:

```
JSON
```

```
{
 "amount":5000
}
```

## Enterprise Service Bus (ESB)

An ESB is middleware that connects services.

Architecture:

```
Service A
    |
    |
    ▼
   ESB
    |
    |
    ▼
Service B
```

ESB handles:

- Message routing
- Data transformation
- Security
- Logging

Examples:

- Mule ESB
- IBM Integration Bus

---

## SOA vs Monolithic Architecture

| Monolithic | SOA |
|---|---|
| One large application | Multiple services |
| Tightly coupled | Loosely coupled |
| Difficult reuse | High reuse |
| Single deployment | Independent service deployment |
| Less flexible | More flexible |

---

## SOA vs Microservices

SOA and Microservices are related but different.

| SOA | Microservices |
|---|---|
| Older architecture style | Modern architecture approach |

| SOA | Microservices |
|---|---|
| Larger enterprise services | Smaller focused services |
| Often uses ESB | Usually uses lightweight communication |
| Services may share databases | Usually each service owns its data |
| SOAP commonly used | REST/events commonly used |

---

# Example Difference

### SOA:

```
Enterprise System

Customer Service
      |
      ESB
      |
Payment Service
      |
      ESB
      |
Order Service
```

### Microservices:

```
Customer Service
      |
      REST/Event
      |
Payment Service
      |
      REST/Event
      |
Order Service
```

# Advantages of SOA

# 1. Reusability

A service can support multiple applications.

Example:

Payment service:

```
Website
Mobile App
ATM System
```

all use the same service.

---

## 2. Easier Maintenance

Changes are isolated.

Example:

Updating payment rules does not require changing the whole application.

---

## 3. Better Integration

SOA helps connect:

- Legacy systems
- New applications
- External partners

---

## 4. Scalability

Individual services can be scaled independently.

Example:

During shopping festivals:

```
Payment Service
      ↑
Scale more servers
```

# Disadvantages of SOA

## 1. Complexity

Managing many services can be difficult.

Problems:

- Service communication
- Monitoring
- Security
- Deployment

---

## 2. Performance Overhead

Communication between services adds network delay.

Example:

Instead of:

```
Function call
```

you have:

```
Network request
      ↓
Service response
```

## 3. Requires Strong Governance

Large organizations need rules for:

- Service design
- Versioning
- Security
- Documentation

---

## SOA in Enterprise Applications

SOA is commonly used in:

- Banking systems
- Insurance platforms
- Government systems
- Healthcare systems
- Airline reservation systems
- Large enterprise software

---

## SOA and API Relationship

An API is a way for services to communicate.

SOA uses APIs to expose services.

Example:

```
Payment Service

API:
POST /payment

Input:
Customer ID
Amount
```

```
Output:
Payment Status
```

## SOA Interview Definition

> **Service-Oriented Architecture (SOA) is an architectural approach where software functionality is organized into independent, reusable services that communicate through standardized interfaces to provide business capabilities.**

---

## Easy Way to Remember

**Monolith:**

> One big application does everything.

**SOA:**

> Break the application into reusable business services.

**Microservices:**

> Break services into smaller, independently deployable services.

**In one sentence:**

**SOA = Build software as a collection of reusable services that communicate through standard interfaces.**

> MapReduce

## MapReduce

**MapReduce** is a programming model used for **processing and analyzing very large amounts of data** by dividing the work into smaller tasks that can run in parallel across multiple machines.

In simple words:

> **MapReduce breaks a big data processing job into two main steps: Map (process and organize data) and Reduce (combine the results).**

It is widely associated with **big data systems**, especially the Apache Hadoop ecosystem.

---

## Why do we need MapReduce?

Imagine a company like an e-commerce platform has billions of records:

```
Orders:
1,000,000,000+ records
```

A single computer may take hours or days to process this data.

Instead of one machine:

```
            Big Data
               |
               ▼
         Single Computer
               |
         Processing takes days
```

MapReduce distributes the work:

```
            Big Data
               |
               ▼
        Split into smaller parts
               |
   -------------------------------
   |              |               |
Machine 1     Machine 2      Machine 3
   |              |               |
Process       Process        Process
   -------------------------------
               |
               ▼
         Combine Results
```

# The Two Main Steps of MapReduce

# 1. Map Phase

The **Map function** processes input data and converts it into intermediate key-value pairs.

The Map step answers:

> "How can I break this data into useful pieces?"

---

# Example: Counting Words

Input:

```
Hello world
Hello Hadoop
```

The Mapper processes each word:

```
Hello → 1
world → 1
Hello → 1
Hadoop → 1
```

Output:

```
(Key, Value)
```

```
Hello, 1
world, 1
Hello, 1
Hadoop, 1
```

## 2. Shuffle and Sort Phase

Before reducing, the system groups similar keys together.

Input from Mapper:

```
Hello, 1
world, 1
Hello, 1
Hadoop, 1
```

After grouping:

```
Hello → [1,1]
world → [1]
Hadoop → [1]
```

This step prepares data for the Reducer.

---

## 3. Reduce Phase

The **Reduce function** combines values with the same key.

It answers:

> "How do I combine these partial results into a final answer?"

Example:

Input:

```
Hello → [1,1]
world → [1]
Hadoop → [1]
```

Reducer output:

```
Hello → 2
world → 1
Hadoop → 1
```

Final result:

```
Hello appeared 2 times
world appeared 1 time
Hadoop appeared 1 time
```

## Complete MapReduce Flow

```
              Input Data
```

```
                |
                ▼

          MAP PHASE

     Record 1 → (Key, Value)
     Record 2 → (Key, Value)
     Record 3 → (Key, Value)

                |
                ▼

          SHUFFLE & SORT

     Group same keys together

                |
                ▼

         REDUCE PHASE

     Combine values

                |
                ▼

          Final Output
```

## Real-World Example 1: Website Traffic Analysis

A website receives billions of visits.

Data:

```
User A visited page X
User B visited page Y
User C visited page X
```

Goal:

> Count visits per page.

---

## Map Phase

Convert visits into:

```
(Page, Count)

X → 1
Y → 1
X → 1
```

## Reduce Phase

Combine:

```
X → [1,1]
Y → [1]
```

Result:

```
X → 2 visits
Y → 1 visit
```

## Real-World Example 2: Sales Analysis

A company wants total sales by product.

Input:

```
Laptop, 50000
Phone, 20000
Laptop, 60000
```

Map:

```
Laptop → 50000
Phone → 20000
Laptop → 60000
```

Reduce:

```
Laptop → 110000
Phone → 20000
```

## MapReduce Architecture

A typical Hadoop MapReduce system:


## Important Components

## 1. Mapper

Responsible for:

- Reading input data
- Processing records
- Producing key-value pairs

Example:

```
Input:
"apple orange apple"

Output:
apple → 1
orange → 1
apple → 1
```

## 2. Reducer

Responsible for:

- Combining values
- Producing final output

Example:

```
apple → [1,1]

Output:
apple → 2
```

## 3. Combiner

A mini-reducer that runs after the Map phase to reduce data transfer.

Example:

Without Combiner:

```
Mapper Output:

apple → 1
apple → 1
apple → 1
```

With Combiner:

```
apple → 3
```

Less data travels through the network.

---

## MapReduce Data Types

MapReduce works mainly with:

### Key

An identifier.

Example:

```
Product Name
User ID
Date
```

### Value

The associated data.

Example:

```
Quantity
Price
Count
```

Example:

```
(Product, Sales)

Laptop → 50000
```

## Advantages of MapReduce

## 1. Handles Huge Data

Can process:

- Terabytes
- Petabytes

of data.

---

## 2. Parallel Processing

Many machines work together.

Example:

```
1 billion records

Machine 1 → 300 million
Machine 2 → 300 million
Machine 3 → 400 million
```

## 3. Fault Tolerance

If one machine fails:

```
Worker 1 ❌

Task moved to another worker
```

The system continues.

---

## 4. Data Locality

Instead of moving huge data across networks:

> Move computation closer to where data exists.

This improves performance.

---

# Disadvantages of MapReduce

## 1. Slow for Real-Time Processing

MapReduce is designed for batch processing.

Example:

Good:

```
Analyze yesterday's sales
```

Not ideal:

```
Real-time stock price updates
```

## 2. High Latency

The Map → Shuffle → Reduce process adds delay.

---

## 3. Complex Programming Model

Developers must design:

- Mapper logic
- Reducer logic

- Data flow

## MapReduce vs Traditional Processing

| Traditional Processing | MapReduce |
|---|---|
| Usually one machine | Multiple machines |
| Sequential processing | Parallel processing |
| Limited data size | Massive data processing |
| Manual scaling | Distributed scaling |

## MapReduce vs Apache Spark

| MapReduce | Apache Spark |
|---|---|
| Disk-based processing | Mostly memory-based processing |
| Slower | Faster |
| Good for batch jobs | Batch + real-time processing |
| Higher latency | Lower latency |

## Where is MapReduce Used?

## Search Engines

Example:

Analyzing billions of web pages:

```
Web pages
    |
MapReduce
    |
Search Index
```

## Social Media

Examples:

- Counting likes
- Analyzing user activity
- Generating recommendations

## Banking

Examples:

- Transaction analysis
- Fraud detection
- Risk calculations

---

## E-commerce

Examples:

- Sales reports
- Customer behavior analysis
- Product recommendations

---

## MapReduce and Big Data

MapReduce is part of the big data ecosystem:

```
Big Data System

        Hadoop
          |
  ---------------
  |              |
HDFS        MapReduce

Storage     Processing
```

- **HDFS** → Stores massive data
- **MapReduce** → Processes massive data

---

## Interview Definition

> **MapReduce is a distributed data processing programming model that divides large-scale computation into two phases: Map, which processes and transforms input data into key-value pairs, and Reduce, which aggregates those intermediate results to produce the final output.**

---

## Easy Way to Remember

**Map = Break and Transform**

Example:

```
Data → Small pieces → Key-value pairs
```

**Reduce = Combine and Summarize**

Example:

```
Key-value pairs → Final result
```

**In one sentence:**

**MapReduce = A way to process massive datasets by splitting work across many computers and combining the results efficiently.**

> Graph Theory

# Graph Theory

**Graph Theory** is a branch of mathematics and computer science that studies **graphs**, which are structures made of **nodes (vertices)** and **connections (edges)**.

In simple words:

> **Graph Theory is the study of objects and the relationships between them.**

A graph represents things (nodes) and how they are connected (edges).

---

# Basic Structure of a Graph

A graph consists of two main parts:

# 1. Vertex (Node)

A **vertex** represents an object or entity.

Examples:

- Person in a social network
- City in a map
- Computer in a network
- Web page on the internet

Example:

```
A    B    C
D    E    F
```

Each letter is a vertex.

## 2. Edge

An **edge** represents a relationship or connection between vertices.

Example:

```
A -------- B
     |
     |
     C
```

Connections are edges.

---

# Mathematical Representation

A graph is usually written as:

$$G = (V, E)$$

Where:

- **V** = Set of vertices
- **E** = Set of edges

Example:

```
V = {A, B, C}

E = {(A,B), (B,C)}
```

Meaning:

Vertices:

```
A, B, C
```

Connections:

```
A connected to B
B connected to C
```

# Real-World Examples of Graph Theory

## 1. Social Networks

Example: Friends on social media

```
Alice ---- Bob
  |
  |
Charlie
```

Nodes:

```
People
```

Edges:

```
Friend relationships
```

Used for:

- Friend suggestions
- Community detection
- Influencer analysis

---

## 2. Google Search

The internet can be represented as a graph.

```
Page A
  |
  |
Page B
  |
  |
Page C
```

Nodes:

```
Web pages
```

Edges:

```
Links between pages
```

Search engines use graph algorithms to rank pages.

---

## 3. Maps and Navigation

Cities:

```
Bangalore ---- Mysore
      |
      |
    Chennai
```

Nodes:

```
Cities
```

Edges:

```
Roads
```

Used for:

- Google Maps
- GPS navigation
- Shortest route finding

---

## 4. Computer Networks

```
Computer A ---- Router
                  |
                  |
              Computer B
```

Nodes:

```
Computers, routers
```

Edges:

```
Network connections
```

## Types of Graphs

## 1. Undirected Graph

Connections have no direction.

Example:

Friendship:

```
Alice -------- Bob
```

If Alice is Bob's friend, Bob is Alice's friend.

Mathematically:

```
(A,B) = (B,A)
```

## 2. Directed Graph (Digraph)

Connections have direction.

Example:

Instagram follow:

```
Alice -----> Bob
```

Alice follows Bob, but Bob may not follow Alice.

Represented as:

```
A → B
```

## 3. Weighted Graph

Edges have values or costs.

Example:

Road distances:

```
Bangalore
     |
    150 km
     |
   Mysore
```

Edge weight:

```
150
```

Used in:

- GPS
- Network routing
- Cost optimization

---

## 4. Unweighted Graph

Edges have no values.

Example:

```
A ---- B
```

Only connection matters.

---

## 5. Connected Graph

Every node can reach another node.

Example:

```
A ---- B ---- C
```

All nodes are connected.

## 6. Disconnected Graph

Some nodes are isolated.

Example:

```
A ---- B

C ---- D
```

Two separate groups exist.

## 7. Cyclic Graph

Contains a cycle.

Example:

```
A ---- B
|      |
|      |
D ---- C
```

Path:

```
A → B → C → D → A
```

## 8. Acyclic Graph

No cycles.

Example:

```
A
|
B
|
C
```

## Important Graph Concepts

## Degree of a Vertex

The number of edges connected to a node.

Example:

```
     B
     |
A ---C---D
```

Degree of C:

```
3
```

because C connects to:

- A
- B
- D

---

## Path

A sequence of connected vertices.

Example:

```
A ---- B ---- C
```

Path:

```
A → B → C
```

## Cycle

A path that starts and ends at the same node.

Example:

```
A → B → C → A
```

## Graph Representation in Computer Science

Computers store graphs mainly in two ways.

---

## 1. Adjacency Matrix

A 2D table showing connections.

Example graph:

```
A ---- B
|
C
```

Matrix:

```
    A B C
A   0 1 1
```

```
B    1 0 0
C    1 0 0
```

Meaning:

```
1 = Connected
0 = Not connected
```

### Advantages:

- Fast edge lookup
- Simple implementation

### Disadvantages:

- Uses more memory

---

## 2. Adjacency List

Stores connected nodes for each vertex.

Example:

Graph:

```
A ---- B
|
C
```

Representation:

```
A → B,C
B → A
C → A
```

### Advantages:

- Memory efficient
- Good for large graphs

### Disadvantages:

- Slower to check specific connections

---

## Graph Traversal Algorithms

Traversal means visiting all nodes in a graph.

Two famous algorithms:

---

# 1. BFS (Breadth-First Search)

BFS explores level by level.

Example:

```
      A
    /   \
   B     C
  /
 D
```

Traversal:

```
A → B → C → D
```

Uses:

- Queue data structure

Applications:

- Shortest path in unweighted graphs
- Social network connections

---

# 2. DFS (Depth-First Search)

DFS explores deeply before backtracking.

Example:

```
      A
    /   \
   B     C
  /
 D
```

Traversal:

```
A → B → D → C
```

Uses:

- Stack/recursion

Applications:

- Maze solving
- Cycle detection
- Topological sorting

---

# Important Graph Algorithms

## 1. Dijkstra's Algorithm

Finds the shortest path between nodes.

Example:

Finding:

```
Home → Office
```

with minimum distance.

Used in:

- GPS
- Network routing

---

## 2. Bellman-Ford Algorithm

Finds shortest paths even with negative weights.

Used in:

- Network optimization

---

## 3. Floyd-Warshall Algorithm

Finds shortest paths between all pairs of nodes.

Example:

```
Every city → Every other city
```

## 4. Minimum Spanning Tree (MST)

Connects all nodes with minimum total cost.

Algorithms:

- Kruskal's Algorithm
- Prim's Algorithm

Used in:

- Network design

- Cable installation

---

## 5. Topological Sorting

Ordering tasks with dependencies.

Example:

Software build:

```
Install OS
     ↓
Install Compiler
     ↓
Build Application
```

Used in:

- Scheduling
- Dependency management

---

# Graph Theory Applications in Computer Science

## 1. Databases

Graph databases store relationships.

Example:

A social network:

```
User
 |
Friend
 |
User
```

Used in:

- Recommendation systems
- Fraud detection

---

## 2. Artificial Intelligence

Graphs are used for:

- Knowledge graphs
- Search algorithms

- Neural networks

---

## 3. Machine Learning

Examples:

- Graph Neural Networks (GNNs)
- Relationship prediction

---

## 4. Operating Systems

Used for:

- Deadlock detection

Example:

```
Process A waits for Resource B

Process B waits for Resource A
```

Forms a cycle.

---

## 5. Compilers

Dependency graphs represent:

- Program dependencies
- Execution order

---

## Graph Theory vs Tree

| Graph | Tree |
|---|---|
| General structure | Special type of graph |
| Can contain cycles | No cycles |
| Can have multiple paths | One path between nodes |
| No fixed root | Usually has a root |

Example:

Tree:

```
    A
   / \
```

```
     B   C
```

Graph:

```
A ---- B
|      |
C ---- D
```

## Graph Theory Interview Definition

> **Graph Theory is the study of mathematical structures consisting of vertices and edges used to represent relationships, connections, and interactions between objects. In computer science, graphs are used for modeling networks, searching, routing, databases, and optimization problems.**

---

## Easy Way to Remember

Think of a graph as:

```
Nodes = Things
Edges = Relationships
```

Examples:

```
People + Friendships = Social Graph

Cities + Roads = Map Graph

Web Pages + Links = Internet Graph
```

**In one sentence:**

**Graph Theory helps computers understand and solve problems involving relationships and connections.**

> State Machine

## State Machine

A **State Machine** (also called a **Finite State Machine — FSM**) is a mathematical and computer science model used to describe a system that can exist in **different states** and changes from one state to another based on **events or conditions**.

In simple words:

> **A State Machine describes how an object behaves by defining its possible states and the rules for moving between those states.**

---

## Basic Idea

Every state machine has:

1. **States** → Different conditions a system can be in.
2. **Events/Inputs** → Things that cause changes.
3. **Transitions** → Rules that move the system from one state to another.
4. **Actions** → Operations performed during a transition.

General structure:

```
          Event
            |
            ▼

Current State -----> New State
            |
            |
         Action
```

## Real-World Example: Traffic Light

A traffic light is a simple state machine.

States:

```
RED
GREEN
YELLOW
```

Transitions:

```
         Timer
           |
           ▼

RED ──────────► GREEN
 ▲               |
 |               |
 |               ▼
YELLOW ◄──── GREEN
```

Rules:

```
RED → GREEN
GREEN → YELLOW
YELLOW → RED
```

The light cannot randomly jump:

```
RED → YELLOW ❌
```

because that transition is not allowed.

---

## Components of a State Machine

# 1. State

A state represents the current condition of a system.

Examples:

### User Account

```
Active
Inactive
Suspended
Deleted
```

### Order System

```
Created
Paid
Shipped
Delivered
Cancelled
```

# 2. Event

An event triggers a state change.

Examples:

```
Payment Received
Button Clicked
Timer Expired
User Login
```

# 3. Transition

A transition defines movement between states.

Example:

```
Order Created

Payment Successful

        ↓

Order Paid
```

# 4. Action

An action happens when transitioning.

Example:

```
Payment Successful
```

```
        ↓

Change Order Status
Send Email
Update Inventory
```

## Example: Online Shopping Order State Machine

An order does not stay in one condition forever.

It moves through states:

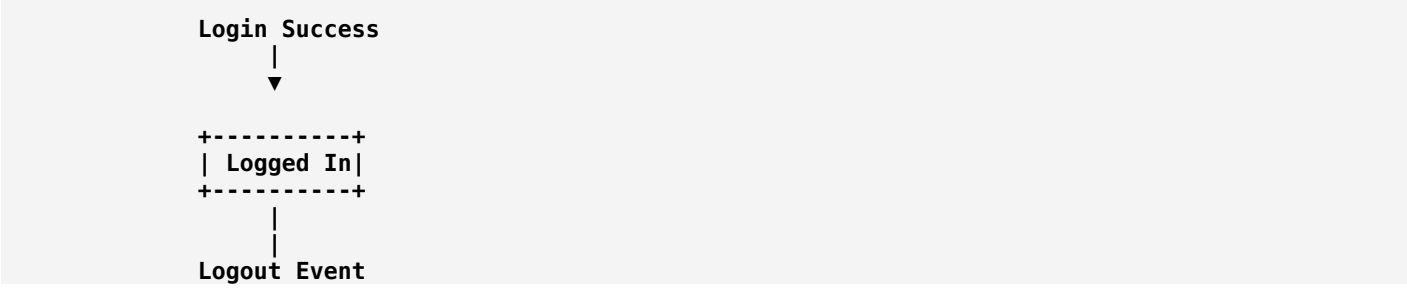
```
        Place Order

             ↓

          CREATED

             ↓

       Payment Done

             ↓

           PAID

             ↓

      Product Shipped

             ↓

         SHIPPED

             ↓

       Customer Receives

             ↓

        DELIVERED
```

Possible cancellation:

```
CREATED
   |
   |
Cancel
   |
   ▼
CANCELLED
```

## State Machine Diagram

Example:



```
              |
              ▼
         +----------+
         | Logged Out|
         +----------+
```

## Types of State Machines

## 1. Finite State Machine (FSM)

The system has a limited number of states.

Example:

ATM machine:

```
Insert Card
     |
     ▼
Enter PIN
     |
     ▼
Select Transaction
     |
     ▼
Withdraw Money
```

The possible states are predefined.

---

## 2. Deterministic Finite Automaton (DFA)

For every state and input, there is exactly one possible next state.

Example:

A password validator:

```
Input: Correct password

Locked
  |
  ▼
Unlocked
```

## 3. Non-Deterministic Finite Automaton (NFA)

A state can have multiple possible transitions.

Used mainly in:

- Pattern matching
- Regular expressions

---

# State Machine in Software Development

State machines are used when objects have **different behaviors depending on their current state**.

---

## Example 1: Login System

States:

```
Logged Out
Logged In
Locked
```

Transitions:

```
Logged Out
     |
Correct Password
     |
     ▼
Logged In


Wrong Password (3 times)
     |
     ▼
Locked
```

Behavior changes depending on state:

Logged In:

```
Allow access
```

Locked:

```
Deny access
```

## Example 2: Media Player

States:

```
Stopped
Playing
Paused
```

Transitions:

```
          Play
Stopped --------> Playing

          Pause
Playing --------> Paused

          Stop
Playing --------> Stopped
```

Actions:

Playing:

```
Continue audio
```

Paused:

```
Keep position
```

Stopped:

```
Reset position
```

## Example 3: Elevator System

States:

```
Ground Floor
First Floor
Second Floor
Moving Up
Moving Down
```

Events:

```
Button Press
Floor Reached
Door Opened
```

The elevator changes state based on events.

---

## State Machine in Programming

A simple implementation:

### Example: Traffic Light (Python)

```python
class TrafficLight:

    def __init__(self):
        self.state = "RED"

    def change(self):
        if self.state == "RED":
            self.state = "GREEN"

        elif self.state == "GREEN":
            self.state = "YELLOW"

        elif self.state == "YELLOW":
            self.state = "RED"
```

Usage:

```
Python
light = TrafficLight()

print(light.state)

light.change()

print(light.state)
```

Output:

```
RED
GREEN
```

## State Machine vs If-Else Logic

Without a state machine:

```
Python
if order == "created":
    do_something()

elif order == "paid":
    do_something_else()

elif order == "shipped":
    another_action()
```

As systems grow, this becomes difficult.

---

With a state machine:

```
Order State

Created
   |
Paid
   |
Shipped
```

The rules are clearer and easier to maintain.

---

## State Machine Applications

## 1. Operating Systems

Used for:

- Process management

Process states:

```
New
 |
Ready
```

```
 |
Running
 |
Waiting
 |
Terminated
```

## 2. Games

Character states:

```
Idle
Running
Jumping
Attacking
Dead
```

Example:

A character cannot attack while dead.

---

## 3. Networking

TCP connection states:

```
CLOSED
 |
LISTEN
 |
ESTABLISHED
 |
CLOSED
```

## 4. Embedded Systems

Examples:

- Washing machines
- Robots
- Traffic controllers

Washing machine states:

```
Idle
 |
Filling Water
 |
Washing
 |
Rinsing
 |
Spinning
```

## 5. Workflow Systems

Examples:

Bank loan approval:

```
Submitted
     |
     ▼
Under Review
     |
     ▼
Approved
     |
     ▼
Disbursed
```

## State Machine vs Flowchart

| State Machine | Flowchart |
|---|---|
| Focuses on states and transitions | Focuses on steps/process |
| Represents behavior over time | Represents algorithm flow |
| Events cause changes | Decisions control flow |
| Used for interactive systems | Used for procedures |

---

## State Machine vs Event-Driven Architecture

They are related:

**State Machine:**

> Defines how something changes state.

Example:

```
Order:
Created → Paid → Shipped
```

**Event-Driven Architecture:**

> Defines how systems communicate using events.

Example:

```
OrderPlaced Event
        |
        ▼
Payment Service
```

A state machine can be triggered by events.

---

## State Machine and Microservices

Microservices often use state machines for managing complex workflows.

Example:

Payment processing:

```
Payment Initiated
        |
        ▼
Processing
        |
   -------------
   |           |
Success     Failed
   |           |
Completed   Retry
```

## Advantages of State Machines

## 1. Clear Behavior

The possible states are explicitly defined.

---

## 2. Easier Debugging

You can identify:

```
Current State:
Payment Processing
```

## 3. Prevents Invalid Operations

Example:

A delivered order cannot return to:

```
Created ❌
```

## 4. Easier Maintenance

Adding new states is simpler.

---

## Disadvantages of State Machines

## 1. Too Many States

Complex systems can create:

```
Hundreds of states
```

which become difficult to manage.

---

## 2. Requires Planning

Incorrect state design can make the system harder to modify.

---

## Interview Definition

> **A State Machine is a computational model that represents a system as a finite set of states and transitions between those states based on events or inputs. It is used to model behavior, workflows, and systems that change over time.**

---

## Easy Way to Remember

Think of a **vending machine**:

```
No Money
    |
Insert Coin
    |
Money Inserted
    |
Select Item
    |
Dispensing
    |
Finished
```

Each condition is a **state**.

Each action is an **event**.

The movement between them is a **transition**.

**In one sentence:**

**State Machine = A model that describes "what state a system is in and what happens when something changes."**

> Concurrency Patterns

## Concurrency Patterns

**Concurrency Patterns** are reusable design techniques used to manage **multiple tasks executing at the same time** in a software system.

In simple words:

> **Concurrency patterns describe common ways to organize, coordinate, and control multiple tasks that run simultaneously.**

They help solve problems like:

- Multiple users accessing a system at the same time
- Processing large amounts of data
- Running background tasks
- Handling many network requests

---

## Concurrency vs Parallelism

These terms are related but different.

## Concurrency

Multiple tasks make progress during the same time period.

Example:

```
Task A: ██---██
Task B: ---██---██
```

A single CPU can switch between tasks.

---

## Parallelism

Multiple tasks execute at exactly the same time using multiple processors.

Example:

```
CPU 1: Task A ████
CPU 2: Task B ████
```

## Why Do We Need Concurrency Patterns?

Without proper design:

```
User Request 1
      |
      ▼
Processing

User Request 2
      |
      ▼
Waiting...
```

The system becomes slow.

With concurrency:

```
User Request 1 ⟶ Worker 1
User Request 2 ⟶ Worker 2
User Request 3 ⟶ Worker 3
```

Multiple tasks are handled efficiently.

---

## Common Concurrency Patterns

---

## 1. Thread Pool Pattern

## Idea

Instead of creating a new thread for every task, maintain a pool of reusable threads.

Example:

Without thread pool:

```
Request 1 → Create Thread
Request 2 → Create Thread
Request 3 → Create Thread
```

Problem:

- Expensive
- Too many threads

---

With thread pool:

```
          Task Queue

              |
              ▼

+---------+---------+---------+
|Thread 1 |Thread 2 |Thread 3 |
+---------+---------+---------+

              |
              ▼

          Execute Tasks
```

## Real-world Example

A web server receives thousands of requests.

Instead of:

```
10000 requests = 10000 threads
```

It uses:

```
10000 requests
        |
        ▼
Thread Pool (100 threads)
```

## Used in:

- Web servers
- Database systems
- Application servers

---

## 2. Producer-Consumer Pattern

## Idea

One or more processes produce data, and others consume it.

Structure:

```
Producer
    |
    ▼
Queue
    |
    ▼
Consumer
```

## Example: Food Delivery System

Producer:

```
Restaurant prepares order
```

Queue:

```
Order Queue
```

Consumer:

```
Delivery worker picks order
```

## Software Example

Logging system:

```
Application
    |
    ▼
Log Queue
    |
    ▼
Log Writer
```

The application does not wait for logs to be written.

---

## 3. Future/Promise Pattern

## Idea

A task starts now, but the result will be available later.

Example:

```
Start Download

      |
      |
      ▼

Continue other work

      |
      ▼

Get Download Result
```

## Example

A website loads:

- User profile
- Recommendations
- Notifications

at the same time.

```
        Request

          |
   ---------------
   |      |      |
Profile  Ads   Recommendations

          |
          ▼

      Combine Results
```

# 4. Async/Await Pattern

## Idea

Allows code to perform tasks asynchronously without blocking execution.

Example:

Normal:

```
Download File
(wait)
Process File
```

Async:

```
Start Download
      |
Continue other work
      |
Download Finished
      |
Process File
```

Example:

JavaScript:

```JavaScript
async function loadData() {
    let data = await fetchData();
    display(data);
}
```

Used in:

- Web applications
- APIs
- Network programming

---

# 5. Actor Model Pattern

## Idea

The system is divided into independent actors.

Each actor:

- Has its own state
- Receives messages
- Processes messages independently

Structure:

```
Actor A
   |
 Message
   |
   ▼
Actor B
```

Example:

Online game:

```
Player Actor
Enemy Actor
Weapon Actor
```

Each actor handles its own behavior.

---

Used in:

- Distributed systems
- Real-time applications

Examples:

- Akka
- Erlang systems

---

# 6. Lock-Based Synchronization Pattern

# Idea

Control access to shared resources using locks.

Problem:

Two threads modify the same data:

```
Thread A:
Balance = 1000

Thread B:
Balance = 1000
```

Both update at the same time.

Result:

Incorrect data.

---

Solution:

Use a lock:

```
Thread A
   |
 Lock Resource
   |
 Update Data
   |
 Unlock


Thread B waits
```

Example:

Bank account:

```
Withdraw money
+
Deposit money
```

Both cannot modify balance simultaneously.

---

# 7. Monitor Pattern

## Idea

A monitor combines:

- Shared data
- Locking mechanism
- Condition checking

Example:

A printer shared by many users:

```
Printer Monitor

User A → Request
User B → Wait
User C → Wait
```

Only one user prints at a time.

---

# 8. Semaphore Pattern

## Idea

A semaphore controls how many tasks can access a resource at once.

Example:

A database allows 100 connections.

```
Semaphore = 100

Connection 1 ✓
Connection 2 ✓
...
Connection 100 ✓

Connection 101 waits
```

Used for:

- Connection pools
- Rate limiting

---

## 9. Fork-Join Pattern

## Idea

Split a large task into smaller tasks, execute them concurrently, then combine results.

Flow:

```
              Big Task

                 |
          ----------------
          |      |       |
      Task1   Task2   Task3

          ----------------

                 |
                 ▼

            Combine Result
```

Example:

Sorting a huge list:

```
[10 million numbers]

       Split

[Part 1]
[Part 2]
[Part 3]

       Sort

       Merge
```

Used in:

- Parallel algorithms
- Big data processing

---

# 10. Pipeline Pattern

## Idea

Processing is divided into stages.

Each stage performs one operation.

Example:

Video processing:

```
Input
 |
 ▼
Decode
 |
 ▼
Process
 |
 ▼
Compress
 |
 ▼
Store
```

Each stage can work simultaneously.

---

# 11. Read-Write Lock Pattern

## Idea

Allows:

- Multiple readers
- Single writer

Example:

Database:

Many users:

```
Read account balance
```

can happen together.

But:

```
Update balance
```

requires exclusive access.

---

Structure:

```
Readers:
R1 R2 R3
  |
  ▼
 Shared Access


Writer:
W
 |
 ▼
 Exclusive Access
```

## 12. Event Loop Pattern

## Idea

A single thread handles many tasks by processing events.

Flow:

```
Events Queue

Request 1
Request 2
Request 3

       |
       ▼

Event Loop

       |
       ▼

Callbacks
```

Used in:

- Node.js
- Browser JavaScript

---

## Real-World Example: E-commerce Checkout

A checkout process may use multiple concurrency patterns.

```
Customer Checkout

        |
        ▼

Order Service
        |
        |
 ------------------------
 |          |           |
Payment  Inventory   Notification
```

```
Async Tasks
      |
      ▼

Combine Results
```

Possible patterns:

- Thread pool → Handle users
- Producer-consumer → Process orders
- Future → Payment response
- Lock → Update inventory safely

---

# Concurrency Problems

## 1. Race Condition

Two tasks access shared data incorrectly.

Example:

```
Bank Balance = 1000

Thread A withdraws 500
Thread B withdraws 500

Wrong result:
Balance = 500
```

## 2. Deadlock

Two tasks wait forever.

Example:

```
Thread A holds Resource 1
waiting for Resource 2

Thread B holds Resource 2
waiting for Resource 1
```

Both stop.

---

## 3. Starvation

A task never gets resources.

Example:

```
High priority tasks
keep running
```

```
Low priority task
never executes
```

## Concurrency Patterns in Different Areas

## Web Servers

Used:

- Thread pools
- Async I/O
- Event loops

---

## Databases

Used:

- Locks
- Transactions
- Semaphores

---

## Operating Systems

Used:

- Scheduling
- Processes
- Threads

---

## Distributed Systems

Used:

- Actor model
- Message queues
- Event-driven processing

---

## Concurrency Patterns vs Parallel Programming

| Concurrency Patterns | Parallel Programming |
|---|---|
| Managing multiple tasks | Running tasks simultaneously |
| Focuses on coordination | Focuses on speed |
| Handles resource sharing | Uses multiple processors |

## Interview Definition

> **Concurrency patterns are reusable software design approaches that help developers manage multiple tasks executing at the same time by providing strategies for communication, synchronization, resource sharing, and task coordination.**

## Easy Way to Remember

| Pattern | Simple Meaning |
|---|---|
| Thread Pool | Reuse workers |
| Producer-Consumer | Create and process through a queue |
| Future/Promise | Result available later |
| Async/Await | Do not block while waiting |
| Actor Model | Independent objects communicate by messages |
| Semaphore | Limit access |
| Lock | Protect shared data |
| Pipeline | Process in stages |
| Fork-Join | Split and combine work |

**In one sentence:**

**Concurrency Patterns = Proven ways to make multiple tasks work together safely and efficiently.**

12-Factor App Methodology

## 12-Factor App Methodology

The **12-Factor App Methodology** is a set of best practices for building **modern, scalable, and maintainable cloud-based applications**.

It was introduced in 2011 by engineers at Heroku to help developers build applications that work well in cloud environments.

In simple words:

> **The 12-Factor App is a guideline for designing applications that are easy to deploy, scale, configure, and maintain.**

It is commonly used in:

- Cloud applications
- Microservices
- SaaS products
- Containerized applications
- DevOps environments

---

## Why Do We Need 12-Factor Apps?

Traditional applications often had problems:

### Problem 1: Configuration mixed with code

Example:

```
Python
database_password = "mypassword123"
```

Changing environments required changing code.

---

### Problem 2: Difficult deployment

Application works on developer machine:

```
"It works on my computer!"
```

but fails in production.

---

### Problem 3: Scaling problems

Adding more servers becomes complicated.

---

The 12-Factor approach solves these problems by making applications:

- Portable
- Cloud-friendly
- Independently deployable
- Easy to scale

---

## The 12 Factors

---

# 1. Codebase

## Principle:

> One codebase tracked in version control, many deployments.

A single application should have one source repository.

Example:

```
Git Repository

        |
 ---------------
 |              |
Development   Production
```

Same codebase, different environments.

---

## Good Practice:

Use:

- Git
- GitHub
- GitLab
- Bitbucket

Avoid:

```
Application_v1_final.zip
Application_v2_final.zip
```

# 2. Dependencies

## Principle:

> Explicitly declare and isolate dependencies.

An application should clearly specify the libraries it needs.

Example:

Python:

```
requirements.txt
```

Contains:

```
Django==5.0
requests==2.31
```

Java:

```
pom.xml
```

Node.js:

```
package.json
```

Benefits:

- Same environment everywhere
- Easier deployment
- Fewer compatibility issues

---

# 3. Config

# Principle:

> Store configuration in environment variables, not code.

Bad:

```
Python
DATABASE_URL = "mysql://localhost/mydb"
```

Good:

```
Python
DATABASE_URL = os.environ["DATABASE_URL"]
```

Configuration examples:

- Database URL
- API keys
- Passwords
- Environment settings

---

Example:

Development:

```
DATABASE=dev_database
```

Production:

```
DATABASE=production_database
```

Same code, different configuration.

---

## 4. Backing Services

## Principle:

> Treat external services as attached resources.

A database is not part of your application code.

Example:

```
Application
     |
     |
     ▼
Database Service
```

Examples:

- MySQL
- PostgreSQL
- Redis
- Amazon S3
- Email services

---

The application should be able to switch:

```
Local Database
        ↓
Cloud Database
```

without code changes.

---

## 5. Build, Release, Run

## Principle:

Separate application lifecycle stages.

Three stages:

```
Source Code
    |
    ▼
Build
(Compile + Dependencies)
    |
    ▼
Release
(Code + Config)
    |
    ▼
Run
(Application Execution)
```

Example:

A deployment should not modify code while running.

---

## 6. Processes

## Principle:

> Execute the app as one or more stateless processes.

A process should not store important data locally.

Bad:

```
User session stored in server memory
```

Problem:

If server crashes:

```
Session Lost
```

Good:

```
Application Server
        |
External Storage
(Database/Redis)
```

## 7. Port Binding

## Principle:

> Export services through port binding.

The application should provide its own web server.

Example:

```
Application
Runs on:
localhost:8080
```

Instead of depending on a separate server.

---

Example:

Node.js:

```
app.listen(3000)
```

## 8. Concurrency

## Principle:

> Scale applications through processes.

Instead of making one process bigger:

```
One huge server
```

Use multiple processes:

```
Application
 |
 -----------------
 |        |        |
Process Process Process
```

Example:

A web application:

```
User Requests
       |
       ▼
10 Application Instances
```

## 9. Disposability

## Principle:

> Processes should start quickly and shut down gracefully.

Applications should handle:

- Crashes
- Restarts
- Deployments

---

Example:

Cloud platforms may restart containers anytime.

Good application:

```
Start → Ready quickly

Stop → Save work safely
```

# 10. Dev/Prod Parity

## Principle:

> Keep development, testing, and production environments similar.

Bad:

Developer:

```
Windows + Local Database
```

Production:

```
Linux + Cloud Database
```

Many surprises.

---

Good:

```
Development
     |
     |
Testing
     |
     |
Production
```

Similar environments.

---

# 11. Logs

## Principle:

> Treat logs as event streams.

Application should not manage log files.

Bad:

```
app.log
error.log
backup.log
```

Good:

Application writes logs:

```
stdout
 |
 ▼
Logging System
```

Examples:

- Elasticsearch
- Splunk
- CloudWatch

---

Example log:

```
2026-08-04 User login successful
```

# 12. Admin Processes

## Principle:

> Run administrative tasks as one-off processes.

Examples:

- Database migrations
- Data cleanup
- Reports

---

Example:

Normal application:

```
Web Process
```

Admin task:

```
Migration Process
```

Run separately.

## Complete 12-Factor Summary

| Factor | Meaning |
|---|---|
| 1. Codebase | One codebase, many deployments |
| 2. Dependencies | Declare dependencies explicitly |
| 3. Config | Store configuration separately |
| 4. Backing Services | Treat services as resources |
| 5. Build/Release/Run | Separate deployment stages |
| 6. Processes | Keep processes stateless |
| 7. Port Binding | Export services through ports |
| 8. Concurrency | Scale using processes |
| 9. Disposability | Fast startup and shutdown |
| 10. Dev/Prod Parity | Keep environments similar |
| 11. Logs | Treat logs as streams |
| 12. Admin Processes | Run maintenance separately |

## 12-Factor App Example: Online Shopping System

A modern e-commerce application:

```
                Users

                  |
                  ▼

          Web Application

                  |
       -----------------------
       |          |          |
    Database    Cache      Payment API


Configuration:
Environment Variables

Logs:
Central Logging System

Scaling:
Multiple App Instances
```

It follows:

- Stateless processes
- External databases
- Environment-based configuration
- Independent scaling

---

## 12-Factor App and Microservices

They work well together.

Microservices need:

- Independent deployment
- External configuration
- Stateless services
- Easy scaling

Example:

```
Order Service

     |
     |
12-Factor Principles

     |
     ▼

Cloud Deployment
```

## 12-Factor App and Containers

Containers like Docker fit naturally with 12-factor principles.

Example:

```
Docker Container

    |
    |
Application
    |
Environment Variables
    |
External Services
```

## Advantages of 12-Factor Apps

## 1. Easy Deployment

Applications can move between environments.

## 2. Better Scalability

Add more application instances easily.

## 3. Cloud Friendly

Works well with:

- Kubernetes
- Docker
- Cloud platforms

## 4. Easier Maintenance

Clear separation of:

- Code
- Configuration
- Infrastructure

## 5. Improved Reliability

Applications handle:

- Failures
- Restarts
- Scaling

## Limitations

The 12-factor approach is not perfect for every system.

Challenges:

- Stateful applications need additional design.
- Legacy systems may be difficult to convert.
- Distributed systems add complexity.

## Interview Definition

> **The 12-Factor App methodology is a set of principles for building modern cloud-native applications that are scalable, portable, maintainable, and suitable for continuous deployment by separating code, configuration, dependencies, processes, and infrastructure concerns.**

---

## Easy Way to Remember

Think:

**Code → Dependencies → Config → Services → Deploy → Scale → Monitor**

or simply:

> **12-Factor App = A recipe for building applications that behave well in the cloud.**

Hexagonal Architecture (Ports and Adapters)

## Hexagonal Architecture (Ports and Adapters)

**Hexagonal Architecture** is a software architecture pattern that separates the **core business logic** of an application from external systems like databases, user interfaces, APIs, and third-party services.

It was introduced by Alistair Cockburn around 2005.

In simple words:

> **Hexagonal Architecture keeps the business logic at the center and connects external systems through replaceable adapters.**

The idea is:

- Business rules should not depend on technology.
- External systems should be replaceable.
- Testing should be easier.

---

## The Basic Idea

Traditional application:

```
id="9g0k8s"
       User Interface
             |
             ▼
       Business Logic
             |
             ▼
         Database
```

Problem:

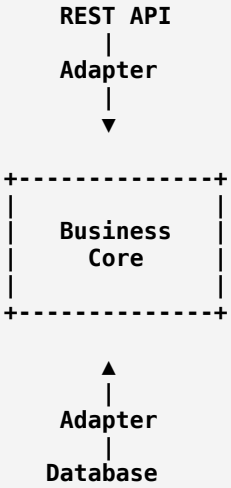
The business logic depends directly on:

- Database technology
- Frameworks
- APIs
- UI

Changing one thing can affect everything.

---

Hexagonal Architecture:



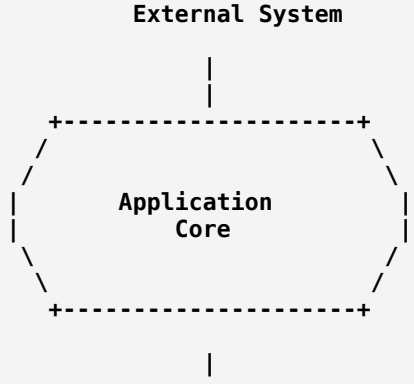

The core does not know whether data comes from:

- PostgreSQL
- MongoDB
- API
- File system

---

## Why is it Called "Hexagonal"?

The architecture is often drawn as a hexagon:



```
                    |

              External System
```

The hexagon is not about having exactly six sides.

It represents:

> **A boundary around the application core.**

---

# Main Components

Hexagonal Architecture has three major parts:

1. **Domain/Application Core**
2. **Ports**
3. **Adapters**

---

# 1. Application Core

The core contains the actual business logic.

Example:

Online shopping system:

```
Order Rules:

- Calculate total price
- Apply discount
- Validate order
- Change order status
```

The core should not know:

```
"Is data stored in MySQL?"
"Is request coming from REST?"
```

It only knows business rules.

---

Example:

```
Java
class Order {

    public void confirmOrder() {
        if(paymentCompleted) {
            status = "CONFIRMED";
        }
    }
}
```

No database code.

No API code.

---

## 2. Ports

Ports are interfaces that define how the outside world communicates with the core.

There are two types:

---

## A. Input Ports (Driving Ports)

These define how external systems use the application.

Examples:

- REST API
- Command line
- User interface
- Message queue

Example:

```
User clicks "Buy"

        |
        ▼

OrderService Port

        |
        ▼

Application Core
```

The port says:

```
createOrder()
cancelOrder()
getOrder()
```

## B. Output Ports (Driven Ports)

These define what the application needs from external systems.

Examples:

- Database
- Email service
- Payment gateway

Example:

```
Application Core
        |
        ▼
Payment Port
        |
        ▼
Stripe / Bank API
```

The core says:

"I need a payment service."

It does not care how payment happens.

---

## 3. Adapters

Adapters connect the outside world to ports.

They translate between:

- External technology
- Application core

---

## Types of Adapters

## Driving Adapters

They send requests into the application.

Examples:

- REST Controller
- Web UI
- CLI
- Message Consumer

Example:

```
HTTP Request
      |
      ▼
REST Adapter
      |
      ▼
Application Port
```

## Driven Adapters

They are used by the application.

Examples:

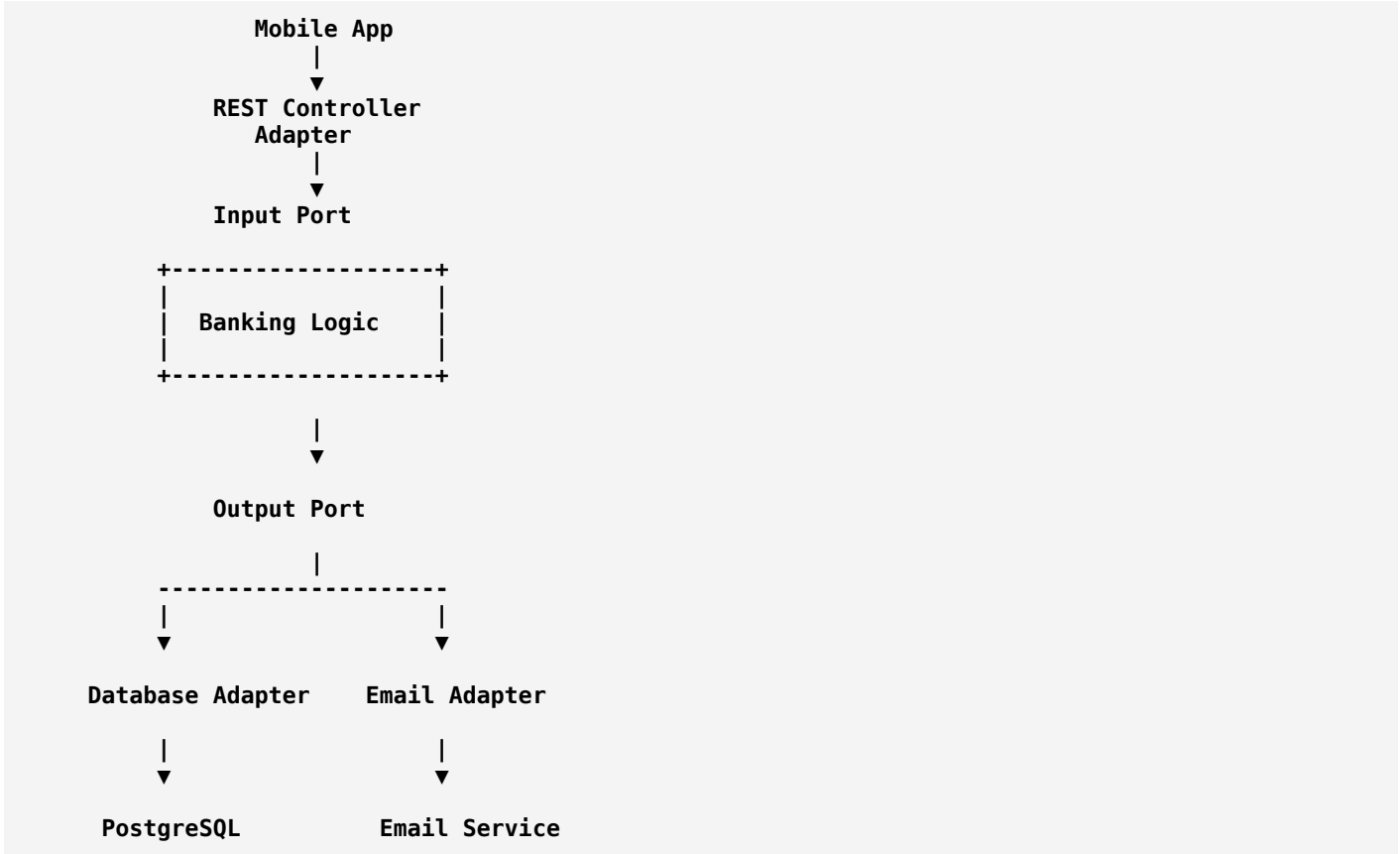
- Database adapter
- Email adapter
- External API adapter

Example:

```
Application Core

      |
      ▼

Database Port

      |
      ▼

PostgreSQL Adapter
```

## Complete Architecture Example

Imagine a banking application.




## Real-World Example: Payment System

Suppose you build a payment application.

The business rule:

```
Process Payment
```

should not depend on:

- PayPal
- Stripe
- Bank API

---

## Core Logic

```
PaymentService
    processPayment()
```

## Port

```
PaymentGateway
    pay(amount)
```

## Adapters

Stripe adapter:

```
StripePaymentAdapter
```

PayPal adapter:

```
PayPalPaymentAdapter
```

Bank adapter:

```
BankPaymentAdapter
```

Now switching:

```
Stripe
   |
   ▼
PayPal
```

does not require changing business logic.

---

# Hexagonal Architecture Example in Code

## Port

```
Java
interface PaymentGateway {

    boolean pay(double amount);

}
```

## Adapter

Stripe implementation:

```
Java
class StripeAdapter implements PaymentGateway {

    public boolean pay(double amount){
        // Stripe API call
        return true;
    }
}
```

## Business Core

```
Java
class CheckoutService {

    private PaymentGateway gateway;

    CheckoutService(PaymentGateway gateway){
        this.gateway = gateway;
    }

    public void checkout(double amount){

        gateway.pay(amount);

    }
}
```

The core only depends on the interface.

---

# Benefits of Hexagonal Architecture

## 1. Easy Testing

Business logic can be tested without:

- Database

- Network
- External APIs

Example:

```
Business Logic

      |
      ▼

Fake Database Adapter
```

## 2. Technology Independence

You can replace:

```
MySQL
  ↓
MongoDB
```

without changing business rules.

---

## 3. Better Maintainability

Code responsibilities are clear:

```
Business Rules
      ≠
Infrastructure
```

## 4. Easier Integration

Adding a new interface is simple.

Example:

Existing:

```
REST API
```

Add:

```
Mobile App API
```

No core changes.

---

## 5. Supports Clean Code

Encourages:

- Dependency inversion
- Separation of concerns
- Loose coupling

---

## Hexagonal Architecture and SOLID Principles

It strongly follows SOLID.

## Single Responsibility Principle

Each component has one job.

Example:

```
Payment Logic
Database Logic
API Logic
```

are separate.

---

## Dependency Inversion Principle

The core depends on abstractions.

Example:

Good:

```
Business Logic
      |
      ▼
Interface
      |
      ▼
Database
```

Bad:

```
Business Logic
      |
      ▼
MySQL Code
```

## Hexagonal Architecture vs Layered Architecture

| Layered Architecture | Hexagonal Architecture |
| --- | --- |
| UI → Business → Database | Core surrounded by adapters |
| Business often depends on database | Database depends on ports |

| Layered Architecture | Hexagonal Architecture |
|---|---|
| Technology influences design | Business drives design |
| Harder to replace components | Easier replacement |

## Hexagonal Architecture vs Clean Architecture

They have very similar goals.

| Hexagonal | Clean Architecture |
|---|---|
| Ports and adapters | Layers and dependency rules |
| Focuses on boundaries | Focuses on dependency direction |
| Created by Alistair Cockburn | Created by Robert C. Martin |

Both promote:

- Independent business logic
- Testability
- Loose coupling

## Where Hexagonal Architecture Is Used

Common in:

## Banking Systems

```
Core:
Transaction Rules

Adapters:
ATM
Mobile Banking
Database
```

## E-commerce

```
Core:
Order Management

Adapters:
Website
Payment Gateway
Inventory System
```

## Healthcare Systems

```
Core:
Patient Rules

Adapters:
Hospital Database
Mobile App
External APIs
```

## Microservices

Each microservice can internally follow hexagonal architecture:

```
Microservice

      API Adapter
          |
          ▼

    Business Core

          |
          ▼

 Database Adapter
```

## Interview Definition

> **Hexagonal Architecture (Ports and Adapters) is a software architecture pattern that isolates the application's business logic from external dependencies by defining ports as interfaces and adapters as implementations, allowing external systems to be replaced without affecting the core application.**

---

## Easy Way to Remember

Think of a **smartphone charger**:

```
Phone (Core)
    |
Charging Port
    |
Different Adapters:
USB-C
Lightning
Wireless Charger
```

The phone does not care which charger is used.

Similarly:

```
Business Logic
       |
      Ports
       |
    Adapters
       |
External Systems
```

**In one sentence:**

**Hexagonal Architecture = Keep the business logic in the center and make everything else replaceable through adapters.**

> Aspect-Oriented Programming (AOP)

# Aspect-Oriented Programming (AOP)

**Aspect-Oriented Programming (AOP)** is a programming paradigm that helps separate **cross-cutting concerns** (features that affect many parts of an application) from the main business logic.

In simple words:

> **AOP allows you to add common functionality like logging, security, and transaction handling to many parts of a program without repeating the same code everywhere.**

---

# Why Do We Need AOP?

Imagine a banking application.

Many methods need:

- Logging
- Authentication
- Authorization
- Transaction management
- Performance monitoring

Without AOP:

```
Java
public void transferMoney(){

    log("Transfer started");

    checkUserPermission();

    processTransfer();

    log("Transfer completed");
}
```

Another method:

```
Java
public void withdrawMoney(){

    log("Withdrawal started");

    checkUserPermission();

    processWithdrawal();
```

```
    log("Withdrawal completed");
}
```

Problems:

- Duplicate code
- Hard to maintain
- Business logic becomes mixed with technical code

---

With AOP:

```
Business Logic

transferMoney()
withdrawMoney()
depositMoney()


        +
        |
        ▼

Cross-Cutting Features

Logging
Security
Transactions
```

The common functionality is applied automatically.

---

## What are Cross-Cutting Concerns?

A **cross-cutting concern** is a feature that affects multiple parts of an application.

Examples:

## Logging

Almost every service needs:

```
"Method started"
"Method completed"
```

## Security

Many operations need:

```
Is user authenticated?
Does user have permission?
```

## Transactions

Database operations need:

```
Begin Transaction

Commit

Rollback
```

## Monitoring

Track:

- Execution time
- Errors
- Usage statistics

---

## AOP Basic Concepts

AOP has several important terms:

---

## 1. Aspect

An **Aspect** represents a cross-cutting concern.

Examples:

```
Logging Aspect

Security Aspect

Transaction Aspect
```

Example:

```
Java
LoggingAspect {

    beforeMethod()
    afterMethod()

}
```

## 2. Join Point

A **Join Point** is a point during program execution where an aspect can be applied.

Examples:

- Method execution

- Object creation
- Exception handling

Example:

```
Java
public void saveUser()
{
    // Join Point
}
```

## 3. Advice

An **Advice** defines what action should be performed and when.

Types:

---

## Before Advice

Runs before a method.

Example:

```
Before:
Check permission

Method:
Delete User
```

## After Advice

Runs after a method completes.

Example:

```
Method:
Save Order

After:
Write log
```

## Around Advice

Runs before and after a method.

Example:

```
Before:
Start Timer

Method:
Process Payment
```

```
After:
Stop Timer
```

## After Returning Advice

Runs after successful execution.

Example:

```
Payment successful

Send receipt
```

## After Throwing Advice

Runs when an exception occurs.

Example:

```
Database Error

Log Error
```

## 4. Pointcut

A **Pointcut** defines where an aspect should be applied.

Example:

Apply logging to all service methods:

```
Java
execution(* service.*.*(..))
```

Meaning:

```
All methods
inside service package
```

## 5. Target Object

The object whose methods are affected by an aspect.

Example:

```
UserService

+
Logging Aspect

=
Enhanced UserService
```

## 6. Weaving

**Weaving** is the process of connecting aspects with application code.

It can happen:

## Compile-time

During compilation.

---

## Runtime

While application is running.

---

## Load-time

When classes are loaded.

---

## AOP Example

Suppose we have:

```
Java
class PaymentService {

    public void makePayment(){

        System.out.println("Payment processed");

    }
}
```

Without AOP:

```
Java
class PaymentService {

    public void makePayment(){

        logStart();

        authenticate();

        processPayment();

        logEnd();

    }
}
```

With AOP:

Business code:

```
Java
class PaymentService {

    public void makePayment(){

        processPayment();

    }
}
```

Aspect:

```
Java
LoggingAspect {

    before():
        logStart();

    after():
        logEnd();

}
```

The logging is added automatically.

## AOP Architecture

```
                Application

                    |
                    ▼

        +----------------------+
        |                      |
        |    Business Logic    |
        |                      |
        +----------------------+

                    |
                    |
              AOP Framework

                    |
        ------------------------
        |           |          |
     Logging   Security   Transactions
```

## AOP in Spring Framework

AOP is heavily used in Spring Framework.

Example:

## Transaction Management

Instead of writing:

```
Java
beginTransaction();

saveData();

commit();
```

everywhere:

Spring allows:

```
Java
@Transactional
public void saveOrder(){

    database.save(order);

}
```

The framework automatically handles:

- Transaction start
- Commit
- Rollback

---

## Real-World Example: Online Banking

Business method:

```
Transfer Money
```

Required features:

```
              Transfer Money

                    |
       -----------------------------
       |            |              |
   Security      Logging     Transaction
```

AOP handles these separately.

---

Flow:

```
User Request

      |
      ▼

Security Aspect
(Check permission)

      |
      ▼

Transfer Method
```

```
      |
      ▼

Transaction Aspect

      |
      ▼

Database Update

      |
      ▼

Logging Aspect
```

## AOP vs Object-Oriented Programming

| OOP | AOP |
|---|---|
| Organizes code using objects | Organizes code using aspects |
| Focuses on business entities | Focuses on common behaviors |
| Example: Customer, Order | Example: Logging, Security |

They are complementary, not replacements.

---

## AOP vs OOP Example

## OOP:

```
Customer
Order
Payment
Product
```

Represents business objects.

---

## AOP:

```
Logging
Security
Monitoring
Transactions
```

Represents shared behaviors.

---

## Advantages of AOP

## 1. Less Code Duplication

Instead of:

```
100 classes
+
100 logging statements
```

Use:

```
1 Logging Aspect
```

## 2. Cleaner Business Logic

Business code focuses only on business rules.

Example:

Good:

```
Java
processPayment();
```

Not:

```
Java
processPayment();
log();
securityCheck();
transaction();
```

## 3. Easier Maintenance

Change logging rules in one place.

---

## 4. Better Separation of Concerns

Example:

```
Business Logic
       |
       |
-----------------
|               |
Security       Logging
```

# Disadvantages of AOP

## 1. Harder to Understand

Code execution is not always obvious.

A developer may ask:

> "Where did this logging happen?"

because it is injected automatically.

---

## 2. Debugging Difficulty

The actual execution flow can be hidden.

---

## 3. Overuse Can Make Design Complex

Not every feature needs AOP.

---

## Where AOP Is Used

## Enterprise Applications

- Banking
- Insurance
- Healthcare

---

## Web Applications

Examples:

- Authentication
- Request logging
- Performance tracking

---

## APIs

Examples:

- API monitoring
- Rate limiting
- Security checks

---

## Databases

Examples:

- Transactions
- Auditing

---

## AOP and SOLID Principles

AOP supports:

## Single Responsibility Principle

A class focuses only on its main job.

Example:

```
PaymentService

Only payment logic
```

Not:

```
PaymentService

+
Logging
+
Security
+
Monitoring
```

## Separation of Concerns

Different responsibilities stay separate.

---

## AOP and Microservices

Microservices often use AOP for:

- Logging
- Authentication
- Metrics
- Tracing

Example:

```
Order Service
       |
       ▼
Logging Aspect
       |
       ▼
Business Logic
```

## Interview Definition

> **Aspect-Oriented Programming (AOP) is a programming paradigm that separates cross-cutting concerns from core business logic by using aspects, allowing common functionality such as logging, security, and transaction management to be applied across multiple parts of an application without code duplication.**

---

## Easy Way to Remember

Think of a restaurant:

Main work:

```
Cook Food
```

Additional activities:

```
Check security
Record orders
Monitor time
Generate reports
```

AOP says:

> Keep cooking logic separate; attach common activities around it.

**In one sentence:**

**AOP = Add common behaviors to many parts of a program without mixing them into the main business code.**

> Functional Programming

## Functional Programming (FP)

**Functional Programming (FP)** is a programming paradigm where programs are built by composing **functions** and treating computation as the evaluation of mathematical-like functions.

In simple words:

> **Functional Programming focuses on what a program should do rather than how to do it, by using pure functions, immutability, and function composition.**

Instead of changing data step-by-step, functional programming transforms data by passing it through functions.

---

# Basic Idea

## Imperative Programming (How to do)

You tell the computer the steps.

Example:

```python
numbers = [1, 2, 3, 4]

sum = 0

for n in numbers:
    sum = sum + n

print(sum)
```

You describe:

1. Create variable
2. Loop through data
3. Add values
4. Store result

---

## Functional Programming (What to do)

You describe the transformation:

```python
numbers = [1, 2, 3, 4]

result = sum(numbers)

print(result)
```

The focus is on the operation.

---

## Core Principles of Functional Programming

---

## 1. Pure Functions

A **pure function** always produces the same output for the same input and does not change anything outside itself.

Example:

```python
def add(a, b):
    return a + b
```

Input:

```
add(2,3)
```

Always returns:

```
5
```

## Impure Function Example

```python
total = 0

def add(value):
    global total
    total += value
```

Problems:

- Depends on external data
- Changes outside state

---

## Benefits of Pure Functions

- Easy to test
- Predictable behavior
- Fewer bugs

---

## 2. Immutability

**Immutability means data cannot be changed after creation.**

Instead of modifying existing data:

```
Original Data
      |
      ▼
 Modified Data
```

Create a new version:

```
Original Data

      +
      |
      ▼

New Data
```

Example:

Mutable style:

```
Python
numbers = [1,2,3]

numbers.append(4)
```

Original list changes.

---

Functional style:

```
Python
numbers = [1,2,3]

new_numbers = numbers + [4]
```

Original remains:

```
numbers = [1,2,3]

new_numbers = [1,2,3,4]
```

## 3. First-Class Functions

Functions are treated like normal values.

A function can be:

- Stored in a variable
- Passed as an argument
- Returned from another function

Example:

```
Python
def greet():
    return "Hello"

message = greet

print(message())
```

Output:

```
Hello
```

## 4. Higher-Order Functions

A function that:

1. Takes another function as input, or
2. Returns a function

Example:

```
Python
numbers = [1,2,3,4]

result = list(map(lambda x: x*2, numbers))
```

Output:

```
[2,4,6,8]
```

`map()` is a higher-order function.

---

## 5. Function Composition

Combining small functions to create larger functions.

Example:

Two functions:

```
Function A

Input → Process → Output


Function B

Output → Process → Final Output
```

Combined:

```
Input
 |
 ▼
Function A
 |
 ▼
Function B
 |
 ▼
Result
```

Example:

```
Python
```

```
result = uppercase(remove_spaces(text))
```

## 6. Declarative Programming

Functional programming is usually declarative.

You describe the desired result.

Example:

SQL:

```
SQL
SELECT name FROM users;
```

You don't specify:

- How to search
- How to scan memory
- How to retrieve data

The system decides.

---

# Functional Programming Concepts

---

## Map

Transforms every element in a collection.

Example:

Input:

```
[1,2,3]
```

Function:

```
multiply by 2
```

Output:

```
[2,4,6]
```

Python:

```
Python
map(lambda x:x*2,[1,2,3])
```

## Filter

Selects elements based on a condition.

Example:

Input:

```
[1,2,3,4,5]
```

Filter:

```
Keep even numbers
```

Output:

```
[2,4]
```

## Reduce

Combines values into one result.

Example:

Input:

```
[1,2,3,4]
```

Operation:

```
Add all numbers
```

Output:

```
10
```

## Recursion

Functional programming often avoids loops and uses recursion.

Example:

Factorial:

```python
def factorial(n):
    if n == 1:
        return 1

    return n * factorial(n-1)
```

## Functional Programming Example

## Problem:

Find total price of products above $100.

Data:

```python
products = [
    {"name":"Laptop","price":1000},
    {"name":"Mouse","price":20},
    {"name":"Phone","price":500}
]
```

Functional approach:

```python
result = sum(
    map(
        lambda p:p["price"],
        filter(
            lambda p:p["price"] > 100,
            products
        )
    )
)
```

Flow:

```
Products

   |
   ▼

Filter expensive items

   |
   ▼

Extract prices

   |
   ▼

Add values

   |
   ▼

Result
```

# Functional Programming Languages

Some languages are strongly functional:

- Haskell
- Erlang
- Clojure
- F#

Languages that support functional programming:

- JavaScript

- Python
- Java
- C#
- Kotlin
- Scala

---

# Functional Programming in Real Applications

---

## 1. Data Processing

Example:

Processing millions of records:

```
Raw Data
   |
   ▼
Filter
   |
   ▼
Transform
   |
   ▼
Aggregate
```

Used in:

- Analytics
- Big data
- Data pipelines

---

## 2. Web Applications

Example:

User data transformation:

```
Database Data
      |
      ▼
Function
      |
      ▼
```

```
API Response
```

## 3. Concurrent Systems

Functional programming works well with concurrency because:

- Immutable data avoids shared-state problems.
- Pure functions are safer to execute in parallel.

Example:

```
Thread A → Process Data 1

Thread B → Process Data 2
```

No conflict because data is not modified.

---

## 4. Financial Systems

Used for:

- Transaction calculations
- Risk analysis
- Pricing models

Because calculations must be predictable.

---

## Functional Programming vs Object-Oriented Programming

| Functional Programming | Object-Oriented Programming |
|---|---|
| Focuses on functions | Focuses on objects |
| Data is usually immutable | Data is often mutable |
| Behavior is separated from data | Data and behavior are combined |
| Uses function composition | Uses inheritance and polymorphism |
| Good for data transformation | Good for modeling entities |

---

## Example Comparison

## OOP Style

```
BankAccount Object

Methods:
```

```
- deposit()
- withdraw()
- checkBalance()
```

Object controls its state.

---

## Functional Style

```
Account Data

      +

Functions:

deposit(account)

withdraw(account)

calculateBalance(account)
```

Functions transform data.

---

## Advantages of Functional Programming

## 1. Easier Testing

Pure functions are simple to test.

Example:

```
Input → Function → Output
```

## 2. Fewer Side Effects

Less unexpected behavior.

---

## 3. Better Concurrency

Immutable data reduces thread problems.

---

## 4. Reusable Code

Small functions can be combined.

---

## 5. Easier Reasoning

Functions behave predictably.

---

# Disadvantages of Functional Programming

## 1. Learning Curve

Concepts like:

- Recursion
- Higher-order functions
- Monads

can be difficult initially.

---

## 2. Performance Overhead

Creating new immutable objects may require extra memory.

---

## 3. Not Always Natural

Some problems are easier with objects.

Example:

Game characters:

```
Player
Enemy
Weapon
```

are naturally object-oriented.

---

# Functional Programming and Modern Software Design

Many modern applications use a hybrid approach:

Example:

```
Application

     |
     |

OOP:
User
```

```
Order
Payment Objects

     +

Functional:
Data Processing
Validation
Transformation
```

## Functional Programming and Microservices

Functional principles help microservices by encouraging:

- Stateless services
- Predictable behavior
- Easy testing

Example:

```
Request

   |
   ▼

Pure Function

   |
   ▼

Response
```

## Interview Definition

> **Functional Programming is a programming paradigm that treats computation as the evaluation of mathematical functions and emphasizes pure functions, immutability, higher-order functions, and avoiding side effects to create predictable and maintainable software.**

---

## Easy Way to Remember

Functional Programming follows:

```
Input
  |
  ▼
Function
  |
  ▼
Output
```

No hidden changes.

No unexpected side effects.

**In one sentence:**

**Functional Programming = Build software by combining small, predictable functions that transform data without changing existing state.**